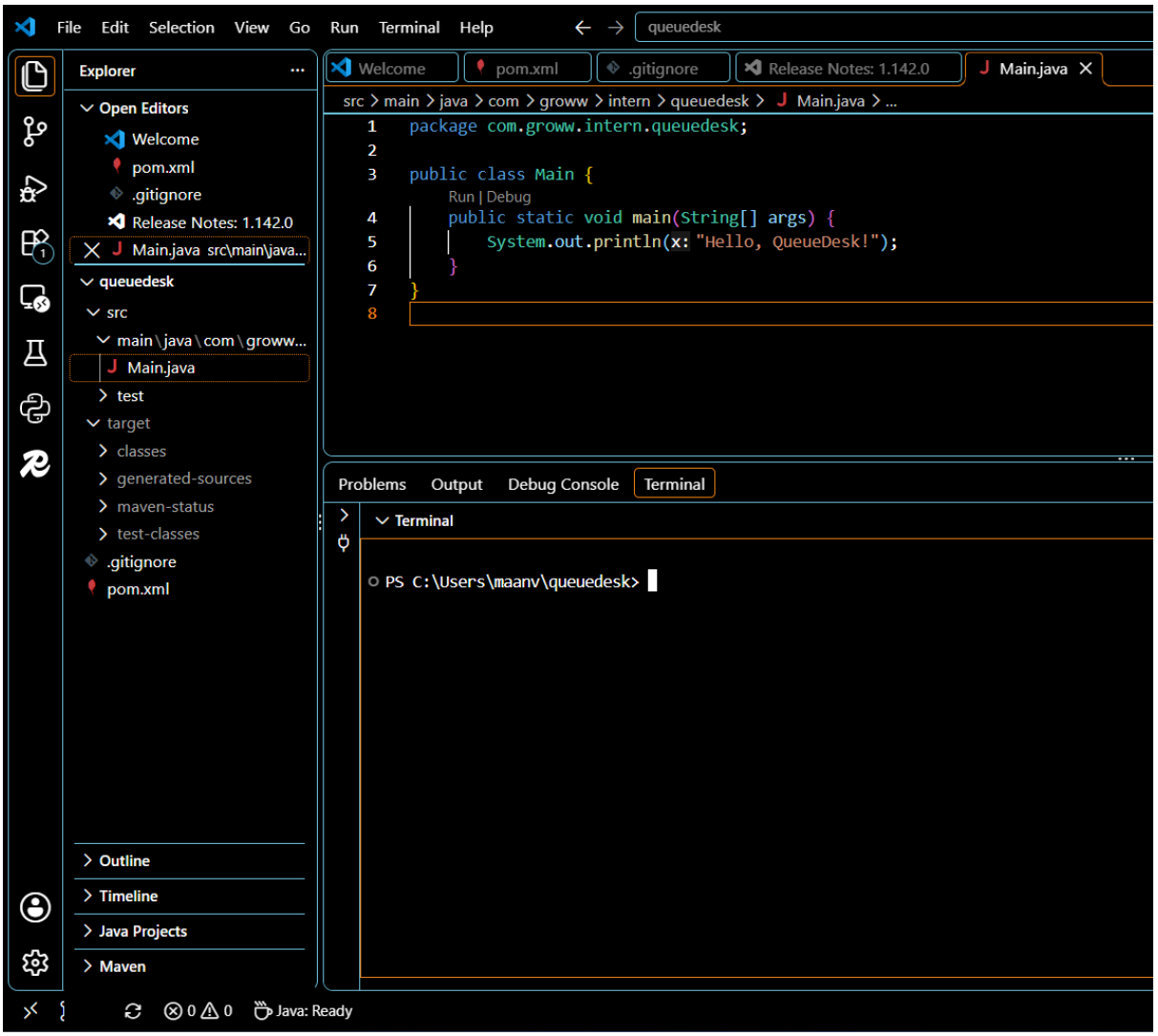
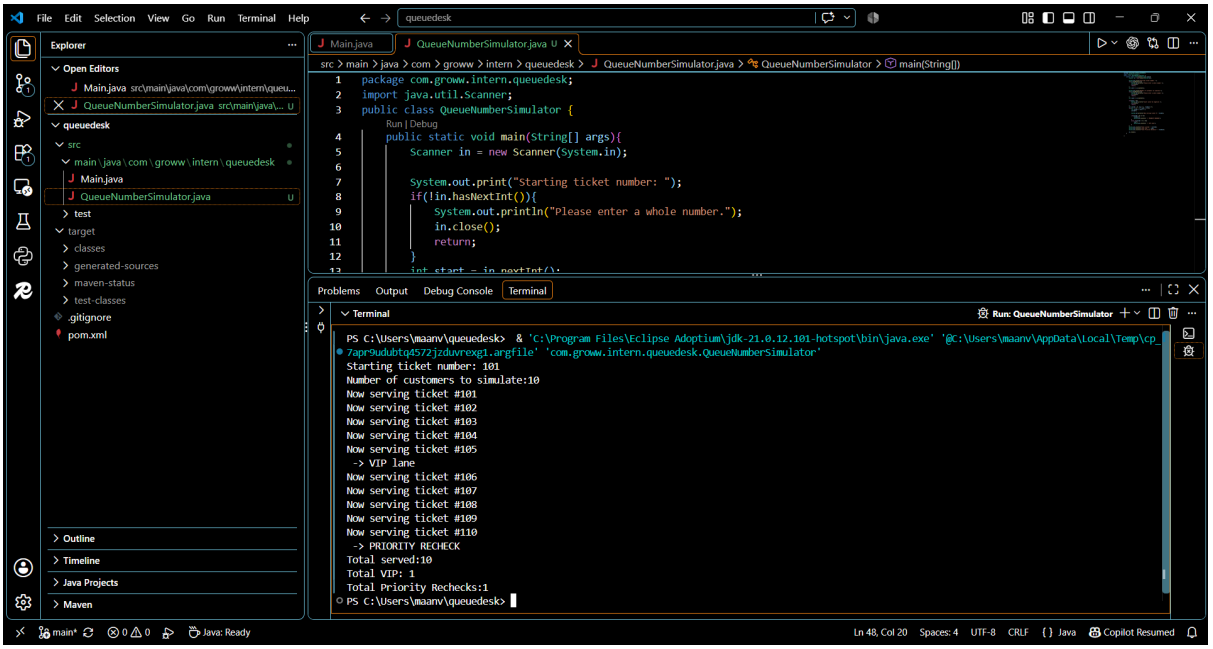
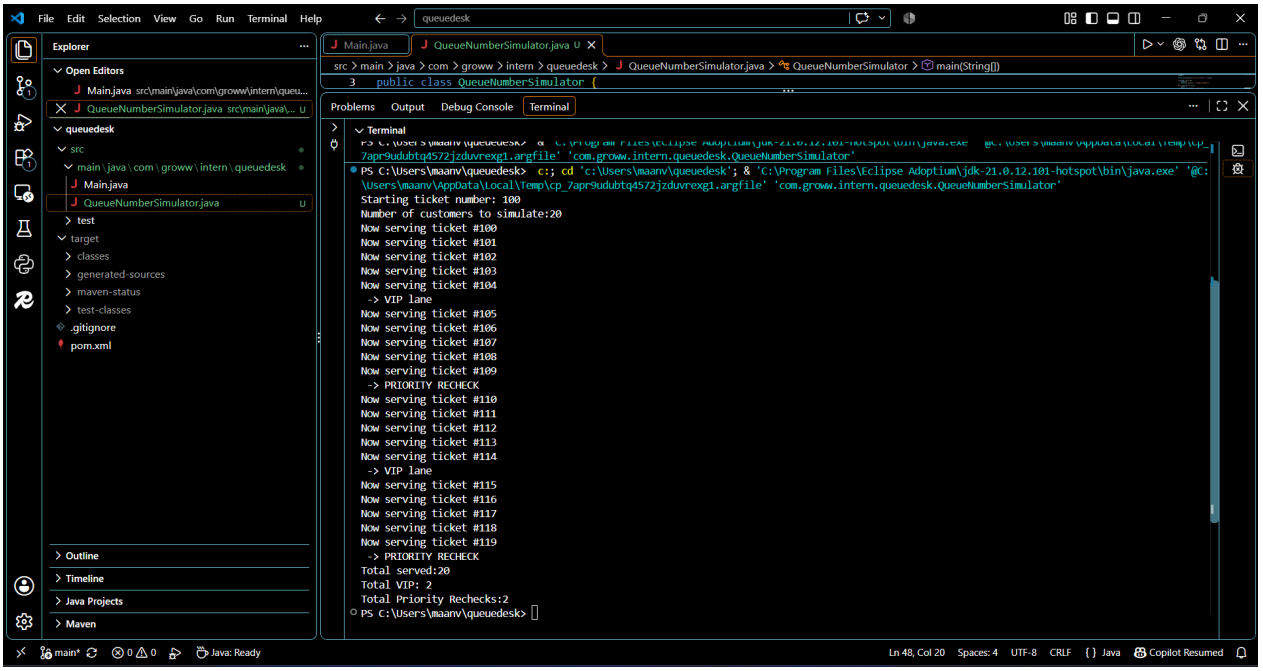

File Edit Selection View Go Run Terminal Help
queuedesk
Explorer
Open Editors
Welcome
pom.xml
.gitignore
Release Notes: 1.142.0
Main.java src\main\java...
queuedesk
src
main \ java \ com \ groww...
Main.java
test
target
classes
generated-sources
maven-status
test-classes
.gitignore
pom.xml
Outline
Timeline
Java Projects
Maven
Welcome
pom.xml
.gitignore
Release Notes: 1.142.0
Main.java
src > main > java > com > groww > intern > queuedesk > Main.java > ...
1 package com.groww.intern.queuedesk;
2
3 public class Main {
Run | Debug
4 public static void main(String[] args) {
5 System.out.println(x: "Hello, QueueDesk!");
6 }
7 }
8
Problems
Output
Debug Console
Terminal
Terminal
PS C:\Users\maanv\queuedesk>
0 0
Java: Ready

BUILD-1

- Build: Queue Number Simulator
  - A single file, no classes of your own beyond the entry point.
  - Read a starting ticket number and a count of customers to simulate, via Scanner.
  - Loop through the count, printing "Now serving ticket #N" for each.
  - Flag every 5th number called as a "VIP lane" ticket and every 10th as a "priority recheck", using the modulus operator and if / else (or switch).
  - Track running totals — served, VIP, priority-recheck — using variables and operators only. No arrays or lists yet.

QueueNumberSimulator.java





BUILD-2

- Build: the Ticket class

- Fields: id, title, description, requester, priority (enum Priority: LOW, MEDIUM, HIGH, CRITICAL), status (enum Status: OPEN, IN_PROGRESS, RESOLVED, CLOSED), createdAt (LocalDateTime.now()).
- A private static int nextId, incremented in every constructor.
- Two constructors: a full one taking title, description, requester, priority; and a short one taking just title and requester, which chains to the full one via this(...) with sensible defaults.
- Getters for every field. Setters only for description, priority, and status.
- An instance method ageInHours() computing time elapsed since createdAt.
- A static method Ticket.totalCreated() returns how many tickets have ever been made, backed by nextId.

Ticket.java

```
/*
REQ 1: CREATE THE FIELDS
A field is a variable declared inside a class that stores an object's
data.
```

```
in java, these variables inside a class are called 'fields'.
id           -> int            -> Unique ticket ID
title        -> String         -> Short summary of the issue
description  -> String         -> Details of the issue
requester    -> String         -> Person who raised the ticket
priority     -> Priority       -> LOW, MEDIUM, HIGH, CRITICAL
status       -> Status         -> OPEN, IN_PROGRESS, RESOLVED, CLOSED
createdAt    -> LocalDateTime  -> Date and time the ticket was created
Priority.java and Status.java are enum that defines the fixed sets of
allowed values.
enum(short for enumeration) is a Java data type used when a variable
should have one value from a fixed set of predefined options. think of
it like a dropdown menu : u can select one of the available options but
can't select a random value.
Example:
enum Priority { LOW, MEDIUM, HIGH, CRITICAL }
enum Status { OPEN, IN_PROGRESS, RESOLVED, CLOSED }
So, instead of using a string, we create an enum. : 1.Restrict values
to predefined options, 2.Prevent invalid values and many typing
mistakes, hence -> makes code more reliable.

REQ-2 : GENERATE TICKET IDs AUTOMATICALLY
private static int nextID = 1;

static -> One shared variable for the entire Ticket class.
nextId -> Stores the next ID to assign.
nextId++ -> Assigns the current ID and then increments it.

when creating a ticket, this.id = nextId++; this assigns the current
counter value to the new ticket and then increments the counter.
this.id means the id belonging to the current ticket.
Example: First ticket gets ID 1, second gets ID 2, third gets ID 3.

REQ 3: CONSTRUCTORS
Constructor = special method that initializes an object.
It runs automatically when we use 'new'.

1. Full constructor:
Accepts title, description, requester and priority.
Automatically generates ID, sets status to OPEN, and records the
current creation time.
```

```
Ticket a = new Ticket(
    "Login fails",
    "Error 500 on submit",
    "asha",
    Priority.HIGH
);

2. Short constructor:
Takes only title and requester.
Accepts only title and requester.
Defaults: description = "", priority = MEDIUM.

Ticket b = new Ticket("Printer jam", "kiran");

this(...) = calls another constructor in the same class. It avoids
repeating initialization code.
It must be the first statement in the constructor.

* The full constructor initializes everything; the short constructor
supplies defaults and reuses the full constructor through this(...).

REQ 4: private AND final
private -> Prevents direct access to a field from outside the class.
final -> Prevents reassignment after initialization.

id, title, requester and createdAt are final.
description, priority and status can be changed.

REQ 5: GETTERS AND SETTERS
Getter -> Returns/acess a field's value.
Setter -> Updates/modifies a field's value.

Getters are provided for all fields.
Setters are provided only for description, priority and status.

REQ 6: AGE OF A TICKET
LocalDateTime.now() -> Gets the current local date and time.
Duration.between() -> Calculates elapsed time.
toHours() -> Returns elapsed whole hours.
```

```
ageInHours() returns how many whole hours have passed since creation.

REQ 7: COUNT TOTAL TICKETS
Ticket.totalCreated() -> Returns the total tickets created in this
program run.
static -> The method belongs to the class, not an individual ticket.
Implementation: nextId - 1.
TicketsCreated  nextId  nextId - 1
        0           1   0
        1           2   1
        2           3   2
        3           4   3

REQ 8: VALIDATION
title == null || title.isBlank()
Rejects a missing or blank title.

Objects.requireNonNull()
Rejects null values for required fields.

trim() -> Removes leading and trailing spaces.

REQ 9: toString()
Provides a readable text representation of a ticket.
Ticket
ID - 1
Title - Login Issue
Priority - HIGH
Status - OPEN
Requester - Asha

Your Java object already contains all this information.
What is the problem?
When we write: System.out.println(t1);



You want Java to print the ticket details in a readable format.

So we tell Java how to display the ticket using toString().
public String toString() {
    return "#" + id + " [" + priority + "/" + status
```

```
            + "] " + title + " (" + requester + ")";
}

This produces:
#1 [HIGH/OPEN] Login issue (Asha)
*/
package com.groww.intern.queuedesk;

import java.time.Duration;
import java.time.LocalDateTime;
import java.util.Objects;

public class Ticket {
    private static int nextId = 1; //shared by all tickets

    private final int id;
    private final String title;
    private String description;
    private final String requester;
    private Priority priority;
    private Status status;
    private final LocalDateTime createdAt;

    //full constructor
    /*can be called using Ticket t1 = new Ticket("Login issue", "Error
500", "Asha", Priority.HIGH); */
    public Ticket(String title, String description, String requester,
Priority priority){
        this.id = nextId++;
        this.title = Objects.requireNonNull(title, "Title"); //syntax:
Objects.requireNonNull(variableToCheck, "Error message if it is null");
        this.description = description == null? "" :
description.trim(); //description is optinal while title, requester and
priority are treated as required fields, this uses a ternary operator
which is short of if-else. If description is null, store an empty
string "".Otherwise, store the description after trimming spaces.
        this.requester = Objects.requireNonNull(requester,
"Requester").trim();
        this.priority = Objects.requireNonNull(priority, "Priority");
        this.status = Status.OPEN;
        this.createdAt = LocalDateTime.now();
```

```
        if(this.title.isBlank()){ //here, we are validating the title
if it is empty or contains only spaces, java throws an error instead of
accepting the ticket.
            throw new IllegalArgumentException("Title cannot be
blank"); //Q:why not use isEmpty()? if we use "" -> return true but if
"  " -> return false. isBlank() returns true for both "" and "   ".
        }
    }

    //short constructor
    /*suppose u wnat to create a ticket but don't want to provide a
description or priority.
    Ticket t2 = new Ticket("printer issue", "kiran");
    the short constructor supplies the value automatically for the
missing fields. like Description : "" , Priority : Priority.MEDIUM
    this(..) -->It calls another constructor in the same class — in
this case, the full constructor. So, rather than repeating all the
initialization code, Java reuses the full constructor.
    this(...) must be the first statement inside the constructor. */

    public Ticket(String title, String requester){
        this(title, "", requester, Priority.MEDIUM);
    }

    //Getters
    public int getId(){
        return id;
    }
    public String getTitle(){
        return title;
    }
    public String getDescription(){
        return description;
    }
    public Priority getPriority(){
        return priority;
    }
    public Status getStatus(){
        return status;
    }
```

```
    public LocalDateTime getCreatedAt(){
        return createdAt;
    }

    //Setters - only for description, priority and status. id, title,
requester and createdAt are final and cannot be changed after
initialization.
    public void setDescription(String description){
        this.description = description == null? "" :
description.trim();
    }
    public void setPriority(Priority priority){
        this.priority = Objects.requireNonNull(priority);
    }
    public void setStatus(Status status){
        this.status = Objects.requireNonNull(status); //eg:
t1.setStatus(Status.IN_PROGRESS);
    }

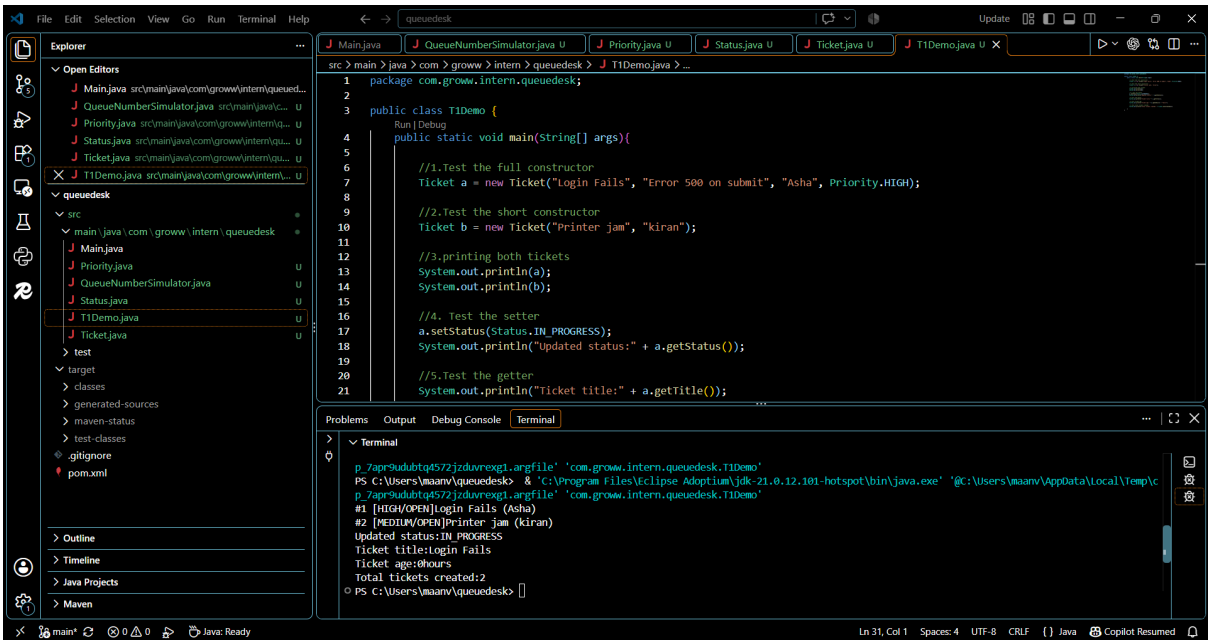
    //returns the age of the ticket in hours, calculated as the
difference between the current time and the ticket's creation time.
    public long ageInHours(){
        return Duration.between(createdAt,
LocalDateTime.now()).toHours();
    }

    //How many tickets have been created? we can use for eg :
System.out.println(Ticket.totalCreated());
    public static int totalCreated(){
        return nextId - 1; //nextId is incremented after each ticket
creation, so the total number of tickets created is nextId - 1.
    }

    //java already knows how to print an object, but it's default
output isn't useful to us, so we write our own toString() method to
provide a more meaningful representation of the ticket object.
    @Override
    public String toString(){
        return "#" + id + " [" + priority + "/" + status + "]" + title
+ " (" + requester + ")";
    }
```

```
}
```

T1.Demo.java


File Edit Selection View Go Run Terminal Help
queuedesk
Update
Explorer
Open Editors
Main.java src\main\java\com\groww\intern\queued...
QueueNumberSimulator.java src\main\java\c... U
Priority.java src\main\java\com\groww\intern\q... U
Status.java src\main\java\com\groww\intern\qu... U
Ticket.java src\main\java\com\groww\intern\qu... U
T1Demo.java src\main\java\com\groww\intern\... U
queuedesk
src
main \ java \ com \ groww \ intern \ queuedesk
Main.java
Priority.java U
QueueNumberSimulator.java U
Status.java U
T1Demo.java U
Ticket.java U
test
target
classes
generated-sources
maven-status
test-classes
.gitignore
pom.xml
Outline
Timeline
Java Projects
Maven
Main.java
QueueNumberSimulator.java U
Priority.java U
Status.java U
Ticket.java U
T1Demo.java U
src > main > java > com > groww > intern > queuedesk > T1Demo.java > ...
1 package com.groww.intern.queuedesk;
2
3 public class T1Demo {
Run | Debug
4 public static void main(String[] args){
5
6 //1.Test the full constructor
7 Ticket a = new Ticket("Login Fails", "Error 500 on submit", "Asha", Priority.HIGH);
8
9 //2.Test the short constructor
10 Ticket b = new Ticket("Printer jam", "kiran");
11
12 //3.printing both tickets
13 System.out.println(a);
14 System.out.println(b);
15
16 //4. Test the setter
17 a.setStatus(Status.IN_PROGRESS);
18 System.out.println("Updated status:" + a.getStatus());
19
20 //5.Test the getter
21 System.out.println("Ticket title:" + a.getTitle());
Problems Output Debug Console Terminal
Terminal
p_7apr9ududbtq4572jzduvrexg1.argfile' 'com.groww.intern.queuedesk.T1Demo'
PS C:\Users\maanv\queuedesk> & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.12.101-hotspot\bin\java.exe' '@C:\Users\maanv\AppData\Local\Temp\c
p_7apr9ududbtq4572jzduvrexg1.argfile' 'com.groww.intern.queuedesk.T1Demo'
#1 [HIGH/OPEN]Login Fails (Asha)
#2 [MEDIUM/OPEN]Printer jam (kiran)
Updated status:IN_PROGRESS
Ticket title:Login Fails
Ticket age:0hours
Total tickets created:2
PS C:\Users\maanv\queuedesk>
main*
0 0
Java: Ready
Ln 31, Col 1
Spaces: 4
UTF-8
CRLF
Java
Copilot Resumed

BUILD-3

- Build: subclasses and a capability
  - Make Ticket abstract; add abstract double estimateEffortHours();.
  - BugTicket extends Ticket — adds severity (enum Severity) and stepsToReproduce.
  - FeatureRequestTicket extends Ticket — adds votes and businessJustification.
  - AccessRequestTicket extends Ticket — adds systemName and accessLevel.
  - Interface Escalatable: void escalate(); boolean isEscalated(); int escalationLevel(); plus one default method, e.g. default String escalationBadge(). Implement it on BugTicket and AccessRequestTicket only.
  - Write printTriageBoard(List<Ticket> tickets) that loops over the base type calling estimateEffortHours() and toString(), then uses if (ticket instanceof Escalatable e) to conditionally show escalation state.

In ticket.java , make these modifications.

```
//Build- T3 Req1: Ticket class is abstract because we don't want to
create a generic ticket, we want to create specific types of tickets
like BugTicket, FeatureRequestTicket, etc. An abstract class cannot be
instantiated directly. It serves as a blueprint for other classes.
public abstract class Ticket
```

```
  //Build-T3 Req2: Add an abstract method to estimate the effort
required to resolve the ticket.Each ticket type estimates it's effort
differently, so we leave the implementation to the subclasses. This
method will be implemented in the subclasses like BugTicket,
FeatureRequestTicket, etc.
    public abstract double estimateEffortHours();
```



When we made the Ticket class abstract, errors were encountered in T1Demo.java as the abstract class cannot be instantiated.

```
package com.groww.intern.queuedesk;
```

```
//build T3 Req2: Severity enum is used to represent the severity of a
ticket. It can be LOW, MEDIUM, HIGH or CRITICAL. This can be used to
prioritize the tickets and to estimate the effort required to resolve
the ticket.
public enum Severity {
    LOW, MEDIUM, HIGH, CRITICAL
}
```

After making it abstract

```
package com.groww.intern.queuedesk;

public class T2Demo {
    public static void main(String[] args) {

        // 1. Create a bug ticket using the full constructor
        BugTicket a = new BugTicket(
            "Login Fails",
            "Error 500 on submit",
            "Asha",
            Priority.HIGH,
            Severity.CRITICAL,
            "Submit the login form"
        );

        // 2. Create another bug ticket
        BugTicket b = new BugTicket(
            "Printer jam",
            "",
            "Kiran",
            Priority.MEDIUM,
            Severity.LOW,
            "Check the printer paper tray"
        );

        // 3. Print both tickets
        System.out.println(a);
        System.out.println(b);

        // 4. Test the status setter
        a.setStatus(Status.IN_PROGRESS);
```

```
        System.out.println("Updated status: " + a.getStatus());

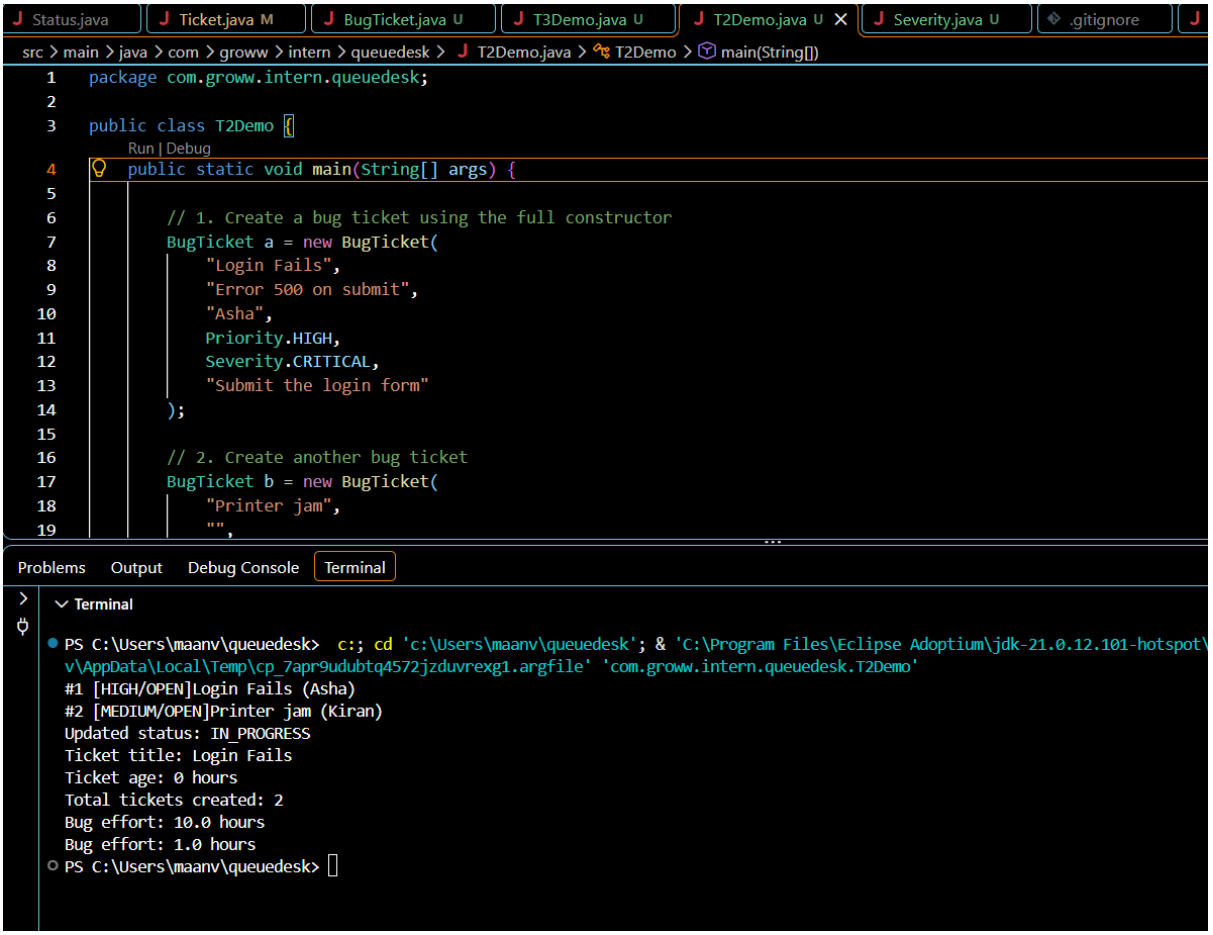
        // 5. Test the title getter
        System.out.println("Ticket title: " + a.getTitle());

        // 6. Test ticket age
        System.out.println("Ticket age: " + a.ageInHours() + " hours");

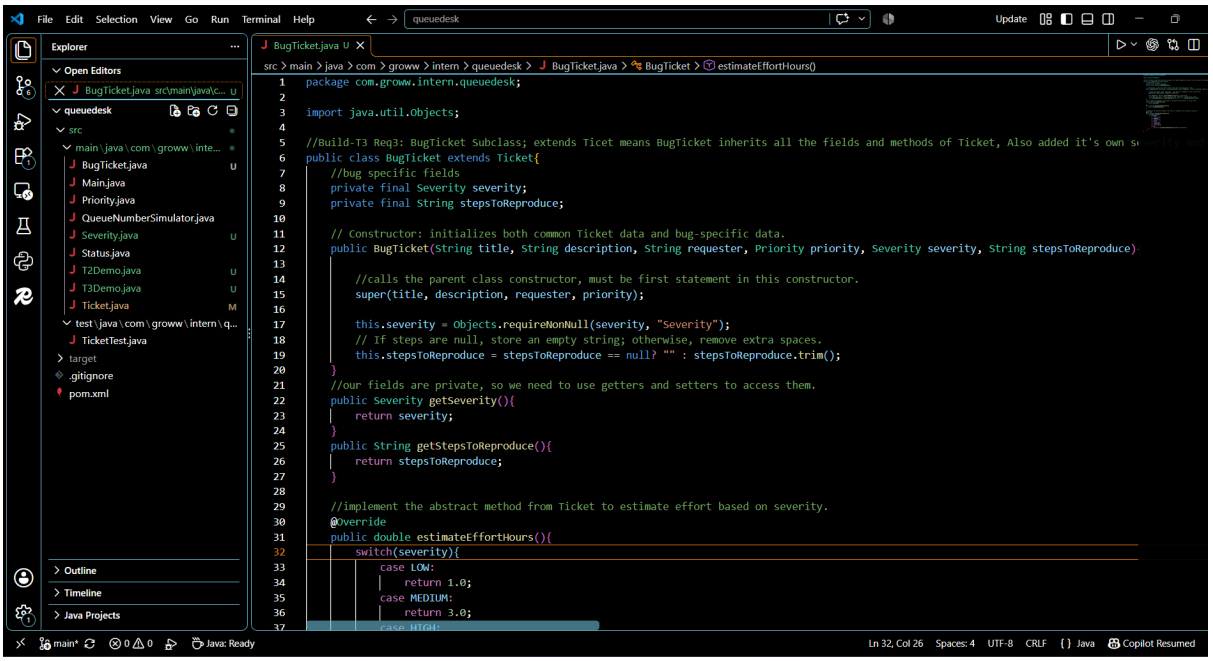
        // 7. Test total tickets created
        System.out.println("Total tickets created: " +
Ticket.totalCreated());

        // 8. Test effort estimation
        System.out.println("Bug effort: " + a.estimateEffortHours() + "
hours");
        System.out.println("Bug effort: " + b.estimateEffortHours() + "
hours");
    }
}
```

T2Demo.java


Status.java
Ticket.java M
BugTicket.java U
T3Demo.java U
T2Demo.java U
Severity.java U
.gitignore
src > main > java > com > groww > intern > queuedesk > T2Demo.java > T2Demo > main(String[])
1 package com.groww.intern.queuedesk;
2
3 public class T2Demo {
Run | Debug
4 public static void main(String[] args) {
5
6 // 1. Create a bug ticket using the full constructor
7 BugTicket a = new BugTicket(
8 "Login Fails",
9 "Error 500 on submit",
10 "Asha",
11 Priority.HIGH,
12 Severity.CRITICAL,
13 "Submit the login form"
14);
15
16 // 2. Create another bug ticket
17 BugTicket b = new BugTicket(
18 "Printer jam",
19 "",
Problems
Output
Debug Console
Terminal
Terminal
PS C:\Users\maanv\queuedesk> c:; cd 'c:\Users\maanv\queuedesk'; & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.12.101-hotspot\
v\AppData\Local\Temp\cp_7apr9udubtq4572jzduvrexg1.argfile' 'com.groww.intern.queuedesk.T2Demo'
#1 [HIGH/OPEN]Login Fails (Asha)
#2 [MEDIUM/OPEN]Printer jam (Kiran)
Updated status: IN_PROGRESS
Ticket title: Login Fails
Ticket age: 0 hours
Total tickets created: 2
Bug effort: 10.0 hours
Bug effort: 1.0 hours
PS C:\Users\maanv\queuedesk>

BugTicket.java




```java
package com.groww.intern.queuedesk;

import java.util.Objects;

//Build-T3 Req3: BugTicket Subclass; extends Ticket means BugTicket
inherits all the fields and methods of Ticket, Also added it's own
severity and stepsToReproduce fields. The constructor initializes all
the fields, including those inherited from Ticket. The
estimateEffortHours() method is implemented to provide a specific
effort estimation for bug tickets based on their severity.
public class BugTicket extends Ticket{
    //bug specific fields
    private final Severity severity;
    private final String stepsToReproduce;

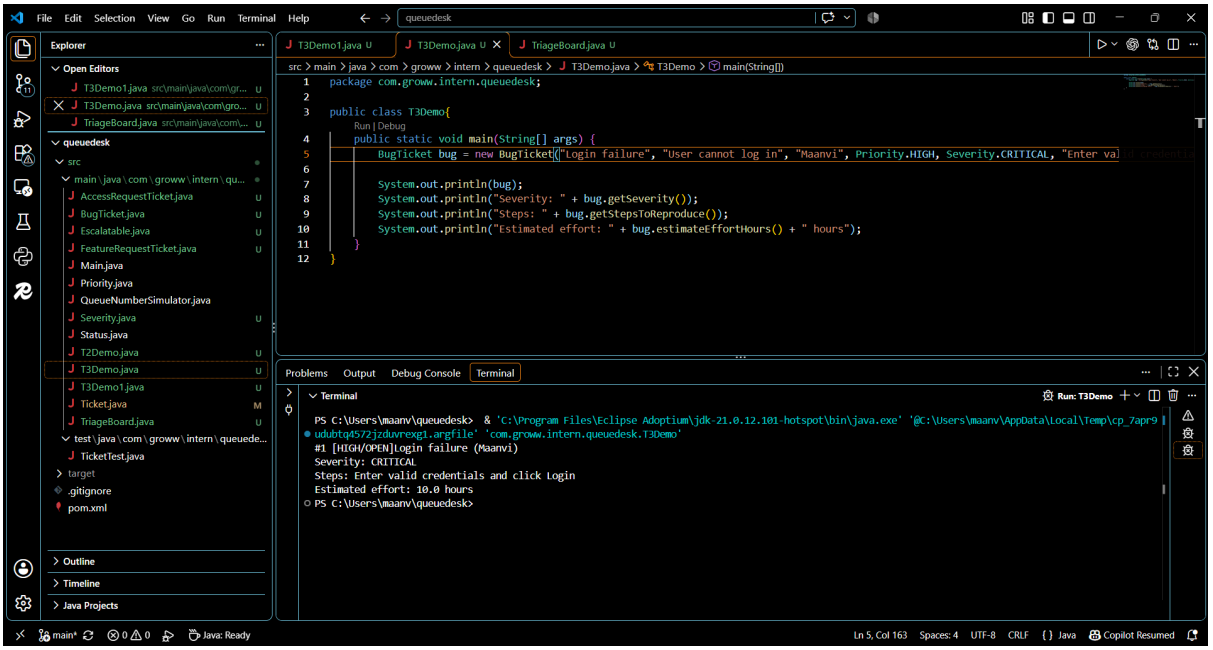
    // Constructor: initializes both common Ticket data and
bug-specific data.
    public BugTicket(String title, String description, String
requester, Priority priority, Severity severity, String
stepsToReproduce){

        //calls the parent class constructor, must be first statement
in this constructor.
        super(title, description, requester, priority);
```

```
        this.severity = Objects.requireNonNull(severity, "Severity");
        // If steps are null, store an empty string; otherwise, remove
extra spaces.
        this.stepsToReproduce = stepsToReproduce == null? "" :
stepsToReproduce.trim();
    }
    //our fields are private, so we need to use getters and setters to
access them.
    public Severity getSeverity(){
        return severity;
    }
    public String getStepsToReproduce(){
        return stepsToReproduce;
    }

    //implement the abstract method from Ticket to estimate effort
based on severity.
    @Override
    public double estimateEffortHours(){
        switch(severity){
            case LOW:
                return 1.0;
            case MEDIUM:
                return 3.0;
            case HIGH:
                return 6.0;
            case CRITICAL:
                return 10.0;
            default:
                throw new IllegalStateException("Unknown severity");
        }
    }
}
```

T3Demo.java



- FeatureRequestTicket extends Ticket — adds votes and businessJustification.
- AccessRequestTicket extends Ticket — adds systemName and accessLevel.
- Interface Escalatable: void escalate(); boolean isEscalated(); int escalationLevel(); plus one default method, e.g. default String escalationBadge(). Implement it on BugTicket and AccessRequestTicket only.

FeatureRequestTicket.java

```
//Build:T3 - Req3: A ticket for requesting a new feature.

package com.groww.intern.queuedesk;

import java.util.Objects;

// votes = how many users want this feature.
// businessJustification = why implementing it is valuable.
public class FeatureRequestTicket extends Ticket {

    // Feature-specific fields
    private int votes;
    private final String businessJustification;

    // Constructor: initializes common and feature-specific fields.
    public FeatureRequestTicket(String title, String description,
String requester, Priority priority, int votes, String
businessJustification) {
```

```
        // Initialize the common Ticket fields.
        super(title, description, requester, priority);

        // Votes cannot be negative.
        if (votes < 0) {
            throw new IllegalArgumentException("Votes cannot be
negative");
        }
        this.votes = votes;

        // Business justification cannot be null.
        this.businessJustification =
Objects.requireNonNull(businessJustification, "Business justification
cannot be null").trim();
    }

    public int getVotes() {
        return votes;
    }

    // Updates votes, ensuring the value is not negative.
    public void setVotes(int votes) {
        if (votes < 0) {
            throw new IllegalArgumentException("Votes cannot be
negative");
        }
        this.votes = votes;
    }

    public String getBusinessJustification() {
        return businessJustification;
    }

    // Provides an effort estimate for a feature request.
    @Override
    public double estimateEffortHours() {
        return 8.0;
    }
}
```

AccessRequestTicket.java

```java
package com.groww.intern.queuedesk;

import java.util.Objects;

// T3 Req4: A ticket for requesting access to a system.
public class AccessRequestTicket extends Ticket implements Escalatable{

    // Fields specific to access requests
    private final String systemName;
    private final String accessLevel;

    // Tracks whether the ticket has been escalated.
    private boolean escalated = false;

    // Tracks the current escalation level.
    private int escalationLevel = 0;

    // Constructor initializes common and access-specific fields.
    public AccessRequestTicket(String title, String description, String
requester, Priority priority, String systemName, String accessLevel) {

        // Initialize common Ticket fields.
        super(title, description, requester, priority);

        // System name and access level cannot be null.
        this.systemName = Objects.requireNonNull(systemName, "System
name cannot be null").trim();

        this.accessLevel = Objects.requireNonNull(accessLevel, "Access
level cannot be null").trim();
    }

    public String getSystemName() {
        return systemName;
    }

    public String getAccessLevel() {
        return accessLevel;
    }
```

```
    //Provides an effort estimate for an access request.
    @Override
    public double estimateEffortHours() {
        return 2.0;
    }
    // Implement the Escalatable interface methods.
    @Override
    public void escalate(){
        escalated = true;
        escalationLevel++;
    }

    @Override
    public boolean isEscalated(){
        return escalated;
    }

   @Override
    public int escalationLevel(){
        return escalationLevel;
    }
}
```

Escalatable.java

```
//An interface defines what a class must be able to do, and not how it
does it.In this case, the Escalatable interface defines the behavior of
tickets that can be escalated. Any class that implements this interface
must provide implementations for the methods defined in the interface.
package com.groww.intern.queuedesk;

// T3 Req5: Defines the behavior of tickets that can be escalated.
public interface Escalatable {

    // Escalate the ticket to a higher level.
    void escalate();

    // Check whether the ticket has been escalated.
```

```
    boolean isEscalated();

    // Return the current escalation level.
    int escalationLevel();

    // Default method shared by classes implementing this interface.
    default String escalationBadge() {
        if (isEscalated()) {
            return "[ESCALATED - Level " + escalationLevel() + "]";
        }
        return "[NOT ESCALATED]";
    }
}
```

- Write printTriageBoard(List<Ticket> tickets) that loops over the base type calling estimateEffortHours() and toString(), then uses if (ticket instanceof Escalatable e) to conditionally show escalation state.

TriageBoard.java

```
package com.groww.intern.queuedesk;

import java.util.List;
/*Imagine QueueDesk has three tickets:
Ticket 1 — BugTicket: Login failure, effort = 10 hours, supports
escalation.
Ticket 2 — FeatureRequestTicket: Add expense report, effort = 8 hours,
does not support escalation.
Ticket 3 — AccessRequestTicket: Database access, effort = 2 hours,
supports escalation.

We want one method that prints all three tickets without writing
separate printing code for each type. That is why printTriageBoard()
exists. */
//T3 Req6: Displays different ticket types on one triage board.
public class TriageBoard {

    //Create a method called printTriageBoard that takes a list of
tickets and prints them all.Why Ticket? becoz all three classes extend
ticket.Accepts a list containing different types of Ticket subclasses.
```

```java
    //List<Ticket> acceps a list of tickets. All 3 classes extend
Ticket, so their objects can be stored in the same list.
    public static void printTriageBoard(List<Ticket> tickets) {

        //loop through tickets- for every ticket in the list, process
one ticket at a time.
        for (Ticket ticket : tickets) {

            //Print the current ticket's details using its toString()
method.
            System.out.println(ticket.toString());

            //Calls the effort-estimation method implemented by the
actual ticket subclass. This is polymorphism: each ticket type can
return a different value.
            //For example, a BugTicket with CRITICAL severity returns
10.0 hours, while an AccessRequestTicket returns 2.0 hours in our
current implementation.
            System.out.println("Estimated effort: " +
ticket.estimateEffortHours() + " hours");

            //Check whether this ticket supports escalation.
            //if (ticket instanceof Escalatable e) — checks whether the
ticket implements Escalatable. If yes, e refers to that ticket through
the interface.
            if (ticket instanceof Escalatable e) {

                //e.escalationBadge() — gets the ticket's escalation
status and prints it.
                System.out.println("Escalation: " +
e.escalationBadge());
            }

            //prints a separator between tickets to make the output
easier to read.
            System.out.println("-------------------------");
        }
    }
}
```

T3Demo1.java

```
package com.groww.intern.queuedesk;

import java.util.ArrayList;
import java.util.List;

//Build T3 Demo: Test the triage board with different ticket types.
public class T3Demo1{
    public static void main(String[] args){

        //Create a BugTicket, parameters: title, description,
requester, priority, severity, stepsToReproduce
        BugTicket bug = new BugTicket("Login failure", "User cannot log
in", "Maanvi", Priority.HIGH, Severity.CRITICAL, "Enter valid
credentials and click Login");

        //Create a FeatureRequestTicket, parameters: title,
description, requester, priority, votes, businessjustification
        FeatureRequestTicket feature = new FeatureRequestTicket("Add
expense reports", "Allow users to download monthly reports", "Asha",
Priority.MEDIUM, 25, "Helps users track their spending");

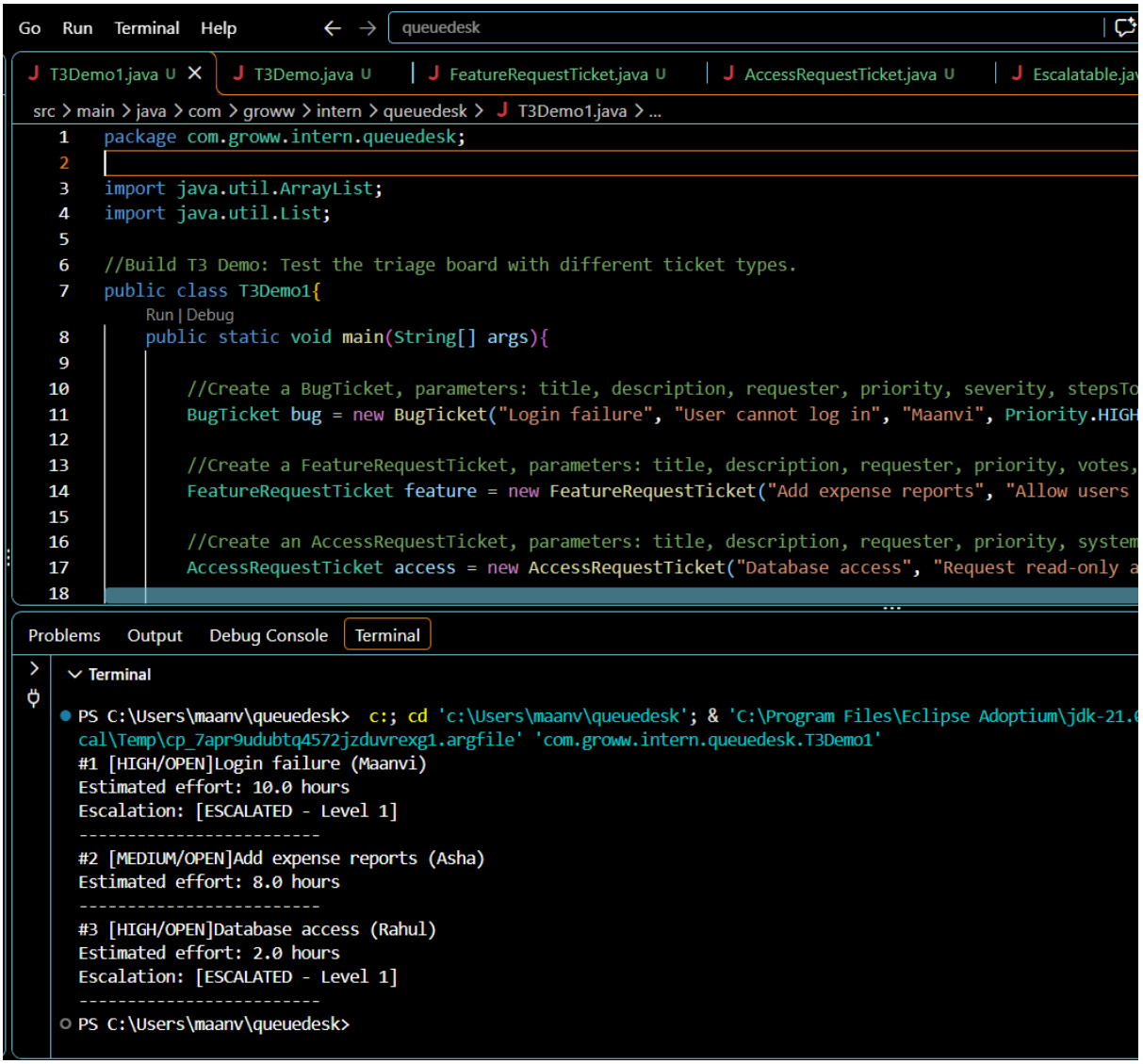
        //Create an AccessRequestTicket, parameters: title,
description, requester, priority, systemName, accessLevel
        AccessRequestTicket access = new AccessRequestTicket("Database
access", "Request read-only access", "Rahul", Priority.HIGH,
"Production Database", "Read-only");

        //Escalate the bug ticket and access request.
        bug.escalate();
        access.escalate();

        //Create one list that can hold all three types of tickets.
Since all three classes extend Ticket, we can store them in a
List<Ticket>.
        List<Ticket> tickets = new ArrayList<>();
        //Add the tickets to the list.
        tickets.add(bug);
        tickets.add(feature);
        tickets.add(access);
```

```
        //Print the triage board with all tickets.
        TriageBoard.printTriageBoard(tickets);
    }
}
/*
// T4Demo.java — CREATE the list
List<Ticket> tickets = new ArrayList<>();
tickets.add(bug);
tickets.add(feature);
tickets.add(access);

// TriageBoard.java — RECEIVE and PROCESS the list
public static void printTriageBoard(List<Ticket> tickets) {
    for (Ticket ticket : tickets) {
        System.out.println(ticket);
    }
} */
```


Go Run Terminal Help
queuedesk
T3Demo1.java U
T3Demo.java U
FeatureRequestTicket.java U
AccessRequestTicket.java U
Escalatable.jav
src > main > java > com > groww > intern > queuedesk > T3Demo1.java > ...
1 package com.groww.intern.queuedesk;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 //Build T3 Demo: Test the triage board with different ticket types.
7 public class T3Demo1{
Run | Debug
8 public static void main(String[] args){
9
10 //Create a BugTicket, parameters: title, description, requester, priority, severity, stepsTo
11 BugTicket bug = new BugTicket("Login failure", "User cannot log in", "Maanvi", Priority.HIGH
12
13 //Create a FeatureRequestTicket, parameters: title, description, requester, priority, votes,
14 FeatureRequestTicket feature = new FeatureRequestTicket("Add expense reports", "Allow users
15
16 //Create an AccessRequestTicket, parameters: title, description, requester, priority, system
17 AccessRequestTicket access = new AccessRequestTicket("Database access", "Request read-only a
18
Problems Output Debug Console Terminal
Terminal
PS C:\Users\maanv\queuedesk> c:; cd 'c:\Users\maanv\queuedesk'; & 'C:\Program Files\Eclipse Adoptium\jdk-21.
cal\Temp\cp_7apr9udubtq4572jzduvrexg1.argfile' 'com.groww.intern.queuedesk.T3Demo1'
#1 [HIGH/OPEN]Login failure (Maanvi)
Estimated effort: 10.0 hours
Escalation: [ESCALATED - Level 1]

#2 [MEDIUM/OPEN]Add expense reports (Asha)
Estimated effort: 8.0 hours

#3 [HIGH/OPEN]Database access (Rahul)
Estimated effort: 2.0 hours
Escalation: [ESCALATED - Level 1]

PS C:\Users\maanv\queuedesk>

BUILD-T4

- Build: repository and triage queue
  - Add assignee (nullable String) and tags (Set<String>) fields to Ticket, with a small setter/adder for each.
  - Make Ticket implements Comparable<Ticket>: order by priority descending, then by createdAt ascending for ties.

Ticket.java

```
import java.util.Set; //Set stores unique elements.
import java.util.HashSet; //HashSet is a collection that implements the Set interface.So, one implementation of Set.
import java.util.Collections; //provides a read-only view of a collection. Collections.unmodifiableSet() returns an unmodifiable view of the specified set. Any attempt to modify the set through this view will result in an UnsupportedOperationException.
```

Then

```
public abstract class Ticket implements Comparable<Ticket>{ //Build-T4 Req 2: Comparable<Ticket> means the ticket class defines how two tickets should be compared for natural ordering.
```

Added,

```
private String assignee; //Build-T4 Req 1: Add a field to store the assignee of the ticket. This field is optional and can be null if the ticket is not assigned to anyone yet.
    private final Set<String> tags = new HashSet<>(); //Build-T4 Req 1: Add a field to store the tags(are labels such as "urgent", "backend") associated with the ticket. This field is a set of strings, which means it can store multiple unique tags. We use HashSet to ensure that each tag is unique and to provide efficient operations for adding, removing, and checking for tags.
```

```
 //Build-T4 Req1: Add methods

    //Getter: returns the assignee of the ticket. If the ticket is not assigned to anyone, it returns null.
    public String getAssignee(){
        return assignee;
    }
```

```
    //Setter: assign the ticket to a specific person.
    public void setAssignee(String assignee){
        this.assignee = assignee;
    }

    //Getter: returns a read-only view of the tags.This prevents
external code from modifying the internal set of tags directly,
ensuring encapsulation.
    public Set<String> getTags(){
        return Collections.unmodifiableSet(tags);
    }

    /*Setter: add a tag to the ticket.
    first replaces the existing tags with the provided set, we validate
each tag through addTag() instead of directly assigning the incoming
set, protecting the object's data..
    */
    public void setTags(Set<String> newTags){
        this.tags.clear(); //remove the existing tags before adding the
new ones.

        if(newTags == null){
            return; //if null is provided, leave the set empty.
        }

        //add each tag through addTag(), which validates the tag before
adding it to the set.
        for(String tag: newTags){ //takes each tag from the newTags set
and passes it to the addTag() method for validation and addition to the
internal tags set.
            this.addTag(tag); //addTag() method checks if the tag is
valid (not null or blank) before adding it to the internal tags set.
            // Checks tag is not null, empty, or just spaces.Then
removes extra spaces using trim() and adds it to the set. If valid,
adds it to the tags set. If invalid, ignores it.
        }
        /*If the incoming set contains " urgent ", addTag() stores it
as "urgent". If it contains "   ", the method throws an exception. */
    }
        //Adder: adds one tag at a time. Set prevents duplicate tags,
while validation prevents null, emty or whitespaces-only tags.
```

```
    public void addTag(String tag){
        if(tag == null || tag.isBlank()){ // If a tag is invalid,
addTag() throws an IllegalArgumentException.
            throw new IllegalArgumentException("Tag cannot be null or
blank");
        }
        //removes leading and trailing spaces from the tag before
adding it to the set.
        tags.add(tag.trim());
    }

    /*Build-T4 Req2: Defines the natural/default sorting order of
tickets.
    ■ Make Ticket implements Comparable<Ticket>: order by priority
descending, then by
    createdAt ascending for ties.
    Means 1.Higher priority tickets comes first: CRITICAL → HIGH →
MEDIUM → LOW.
    2.TIE, If priorities are equal, the older ticket (createdAt) comes
first.*/
    @Override
    public int compareTo(Ticket other){ //the Comparable<Ticket>
interface requires us to implement this method.
    /*Here,
    other= the second ticket being compared to the current ticket
(this).
    this = the current ticket.
    this.priority = priority of the current ticket (this).
    other.priority = priority of the other ticket being compared
(other).
    this.createdAt = creation time of the current ticket (this).
    other.createdAt = creation time of the other ticket being compared
(other).
    ordinal() = the numerical position of that priority in the enum.
    enum Priority {
    LOW, MEDIUM, HIGH, CRITICAL
    }

        //Compare priorities in descending order (CRITICAL → HIGH →
MEDIUM → LOW).
        Priority    ordinal()
```

```
           LOW       0
           MEDIUM    1
           HIGH      2
           CRITICAL  3
        Integer.compare(a, b): Is a shortcut for comparing two integers
instead of writing if-else conditions manually. if(a>b) return 1;
if(a<b) return -1; else return 0;{so negative if first number is
smaller, zero if both numbers are equal and positive if first number is
greater.}
        return Integer.compare(a,b); both expresses the same comparison
logic
        how do we sort? Aesc order: Integer.compare(a,b)
                        Desc order: Integer.compare(b,a)
        Now imagine:
        - Current ticket (this) has LOW → 0
        - Other ticket (other) has CRITICAL → 3
        becomes: Integer.compare(3, 0); The result is positive because
3 > 0.
        Why put other first? Because we want CRITICAL to come before
LOW.
        Reversing the arguments puts higher priorities first.*/
        int priorityComparison =
Integer.compare(other.priority.ordinal(), this.priority.ordinal());

        //if priorities are differnt, return the comparison result.
        if(priorityComparison != 0){
            return priorityComparison;
        }

        //if priorities are equal, compare cration times ie createdAt
in ascending order (older tickets first).
        return this.createdAt.compareTo(other.createdAt);
    }
```

- Write a TicketComparators utility with at least two Comparator<Ticket> built via Comparator.comparing(...).thenComparing(...) — e.g. by assignee then priority, and by status then age.

TicketComparator.java

```
package com.groww.intern.queuedesk;
/*One sorting rule cannot conveniently handle every use case. That's
why we have created TicketComparators.java.
ticket1.compareTo(ticket2) --> Defines the default order: priority
descending, then creation time ascending.
*/
import java.util.Comparator;
//Build-T4 Req3: Provides alternative ways to sort tickets.
public class TicketComparators{
    //Comparator 1: Sort by assignee alphabetically, then by priority
from highest to lowest.
    /*Comparator<Ticket> -  A rule for comparing tickets
    BY_ASSIGNEE_THEN_PRIORITY - The name of the rule
    Ticket::getAssignee - Get the ticket's assignee
    Comparator.nullsFirst(...) - put tickets with no assignee before
tickets with an assignee.
    Comparator.naturalOrder() - Sort names alphabetically
    .thenComparing(...) - If the first comparison is tied, use another
rule
    Ticket::getPriority - Get the ticket's priority
    Priority::ordinal - Get the priority's position in the enum
    .reversed() - Reverse the order so higher enum positions come first
    Comparator.comparingInt(Priority::ordinal).reversed()) - Compare
priorities by their enum position, with higher positions coming first.
*/


    public static final Comparator<Ticket> BY_ASSIGNEE_THEN_PRIORITY =
Comparator.comparing(Ticket::getAssignee,
Comparator.nullsFirst(Comparator.naturalOrder())).thenComparing(Ticket:
:getPriority, Comparator.comparingInt(Priority::ordinal).reversed());

    //Comparator 2 : Sort by status first, then by ticket age(oldest
creation time first).
```

```
    public static final Comparator<Ticket> BY_STATUS_THEN_AGE =
Comparator.comparing(Ticket::getStatus).thenComparing(Ticket::getCreate
dAt);
}
```

- Write a generic class Repository<T extends Ticket> wrapping a private List<T>: add(T), all() (returning an unmodifiable view), findById(int), and groupByAssignee() returning Map<String, List<T>>.

Repository.java

```
package com.groww.intern.queuedesk;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

//Build-T4 Req4: Repository class to store tickets and provide methods.
public class Repository<T extends Ticket> {//bounded type parameter ,
it restricts T so that this repository can only be used for the Ticket
class or any subclass of Ticket class.
    private final List<T> tickets = new ArrayList<>();
   /*private as we don't want other classes to directly access or
modify our internal ticket list.
   Instead, they should use the methods we provide:
   - add() to add tickets.
   - all() to view tickets.
   - findById() to search.
   - groupByAssignee() to group.
    This is an example of Encapsulation.
    List is an interface in java, it represents an ordered collection
of elements , <T> means the list stores objects oof the type
represented by T.
    new ArrayList<>() --> creates an empty list in memory.

    private final List<T> tickets = new ArrayList<>(); creates a
private, initially empty, dynamically resizable list to store tickets
of type T; private provides encapsulation, final prevents reassignment
of the list reference, and List<T> ensures type safety.*/
```

```
    //Req1: add(T ticket) method to add a ticket to the repository.
    public void add(T ticket){ //accepts a ticket and adds it our
internal list ie tickets.
        tickets.add(ticket);
    }

    //Req2: all() method to return all tickets in the repository.
    public List<T> all(){
        return Collections.unmodifiableList(tickets); //returns an
unmodifiable view of the tickets list, preventing external
modification.
    }

    //Req3: findById(int id) method to find a ticket by its ID.
    public T findById(int id){
        for(T ticket : tickets){ //Iterates through each ticket in the
tickets list.
            if(ticket.getId() == id){ //Checks if the ticket's ID
matches the provided ID.
                return ticket; //If a match is found, returns the
ticket.
            }
        }
        return null; //If no ticket with the given ID is found, returns
null.
    }

    //Req4: groupByAssignee() method to group tickets by their
assignee.
    //key = assignee(name-string), value = list of tickets assigned to
that assignee.

    public Map<String, List<T>> groupByAssignee(){
        Map<String, List<T>> groupedTickets = new HashMap<>();
//Creates an empty HashMap to store the grouped tickets.
        for(T ticket : tickets){ //Iterates through each ticket in the
tickets list.
            String assignee = ticket.getAssignee(); //Retrieves the
assignee of the current ticket.
            // groupedTickets.computeIfAbsent(assignee, k -> new
ArrayList<>()).add(ticket); //If the assignee is not already a key in
```

```
the map, creates a new list for that assignee and adds the ticket to
it.

            //If the assignee is not already a key in the map, creates
a new list for that assignee and adds the ticket to it.
            if(!groupedTickets.containsKey(assignee)){ //Checks if the
assignee is already a key in the map.
                groupedTickets.put(assignee, new ArrayList<>()); //If
not, creates a new list for that assignee.
            }
            groupedTickets.get(assignee).add(ticket); //Adds the
current ticket to the list of tickets for that assignee.
        }
        return groupedTickets; //Returns the map of grouped tickets.
    }

}
```

- Build a PriorityQueue<Ticket> triage queue seeded from natural ordering, plus a second one constructed with a Comparator from TicketComparators — offer()/poll() to serve the next ticket either way.

PriorityQ.java

```
package com.groww.intern.queuedesk;
import java.util.PriorityQueue;
//Build-T4 Req5: Manages two ticket triage queues using different
ordering strategies.
public class PriorityQ {

    //Queue1: Natural ordering using Comparable.
    private final java.util.PriorityQueue<Ticket> naturalOrderQueue =
new java.util.PriorityQueue<>(); //Creates a priority queue that orders
tickets based on their natural ordering defined in the Ticket class, ie
Our natural ordering: higher priority first, then older createdAt
first.

    //Queue2: Custom ordering using Comparator.
    private final java.util.PriorityQueue<Ticket> customOrderQueue =
new
java.util.PriorityQueue<>(TicketComparators.BY_ASSIGNEE_THEN_PRIORITY);
```

```
//Creates a priority queue that orders tickets based on a custom
comparator(BY_ASSIGNEE_THEN_PRIORITY comparator) defined in the
TicketComparators class, ie  Our custom ordering: First sorts by
assignee (alphabetically), then by priority (higher first).

    //Adds the same ticket to both queues.
    public void offer(Ticket ticket){
        naturalOrderQueue.offer(ticket);
        customOrderQueue.offer(ticket);
    }

    //removes and returns the next ticket according to natural
ordering, returns null if the queue is empty.
    public Ticket pollNaturalOrder(){
        return naturalOrderQueue.poll();
    }

    //removes and returns the next ticket according to custom ordering,
returns null if the queue is empty.
    public Ticket pollCustomOrder(){
        return customOrderQueue.poll();
    }

    //checks whether the natural order queue is empty.
    public boolean isNaturalOrderQueueEmpty(){
        return naturalOrderQueue.isEmpty();
    }

    //checks whether the custom order queue is empty.
    public boolean isCustomOrderQueueEmpty(){
        return customOrderQueue.isEmpty();
    }

}
```

- Write one generic method: static <T extends Ticket> T oldest(List<T> tickets).

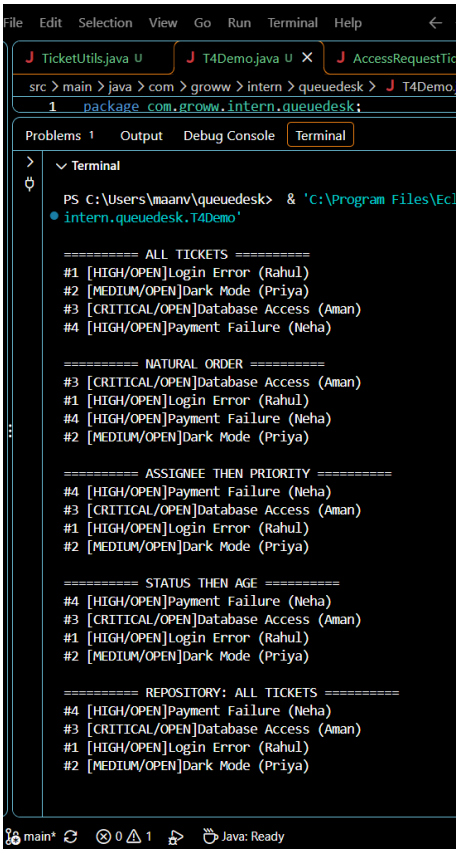
TicketUtils.java

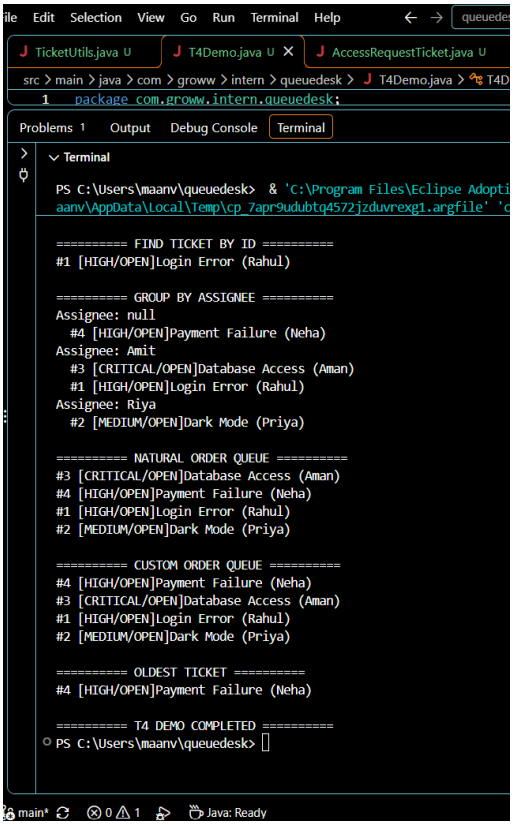
```
package com.groww.intern.queuedesk;
import java.util.List;
//Build-T4 Req6: Utility class containing generic methods for working
with tickets.
public class TicketUtils {
    // Generic method = oldest(List<T> tickets) method to find the
oldest ticket in a list of tickets.
    public static <T extends Ticket> T oldest(List<T> tickets){
//List<T> means the method accepts a list containing objects of type T.
        if(tickets == null || tickets.isEmpty()){ //there is no oldest
ticket if list has no elements.
            return null;
        }
        T oldestTicket = tickets.get(0); //Initially assume the first
ticket is the oldest.
        for(T ticket : tickets){ //Iterates through each ticket in the
list.

if(ticket.getCreatedAt().isBefore(oldestTicket.getCreatedAt())){
//isBefore() Checks if the current ticket was created before the oldest
ticket found so far.
                oldestTicket = ticket; //If so, updates the oldest
ticket to the current ticket.
            }
        }
        return oldestTicket; //Returns the oldest ticket found in the
list(ie return the ticket with the earliest creation time).
    }

}
```

T4Demo.java result





BUILD-T5

- Build: loading, saving, reporting
  - InvalidTicketDataException extends Exception (checked) — thrown while parsing a bad line of tickets.txt: wrong field count, an unparseable Priority, etc. Include the offending line number in the message.
  - TicketNotFoundException extends RuntimeException (unchecked) — thrown by Repository.findById(int) when nothing matches. Callers aren't forced to catch it, but the console app should catch it once, at the top, for a clean error message instead of a stack trace.

Added InvalidTicketDataException

```
package com.groww.intern.queuedesk;
/*
 * Build-T5 Req1: InvalidTicketDataException extends Exception.
Invalid file data
 * This exception will be thrown when a line in ticket.txt contains
invalid ticket data, for eg:
 * 1. wrong field count - Wrong number of fields.
 * 2. an unparseable Priority - Invalid priority such as URGENT when it
is not a valid enum value.
 * 3. Invalid ticket information while parsing.
 *
 * Custom Checked Exception used when a ticket record contains invalid
data while reading tickets.txt.
 * Extending exception makes this a checked exception.
 * Therefore, the caller must either handle it using try-catch block or
declare it using the throws keyword.
 */
public class InvalidTicketDataException extends Exception {
    public InvalidTicketDataException(String message){ //a constructor
that accepts an error message as input ie String message.
        super(message); //calls the parent exception class constructor
and passes the error message to it. This allows us to retrieve the
message later using getMessage().
    }
}
```

And added TicketNotFoundException

```
package com.groww.intern.queuedesk;
/*
Build-T5 Req2: TicketNotFoundException extends RunTimeException.
Ticket ID not found
Custom Unchecked Exception used when a requested ticket cannot be found
in the repository.
Extending RuntimeException makes this an unchecked exception.
Therefore, the caller is not required to handle it using try-catch
block or declare it using the throws keyword.
*/
public class TicketNotFoundException extends RuntimeException {
    public TicketNotFoundException(String message){ //a constructor
that accepts an error message as input ie String message.
        super(message); // calls the parent RuntimeException class
constructor and passes the error message to it. This allows us to
retrieve the message later using getMessage().
    }
}
```

Integrated **TicketNotFoundException** into **Repository.java.**

TicketNotFoundException extends **RuntimeException** — thrown by Repository.findById(int) when nothing matches.

if the requested ticket doesn't exist, the method silently returns null.

Instead, we want it to throw our custom exception with a meaningful message.

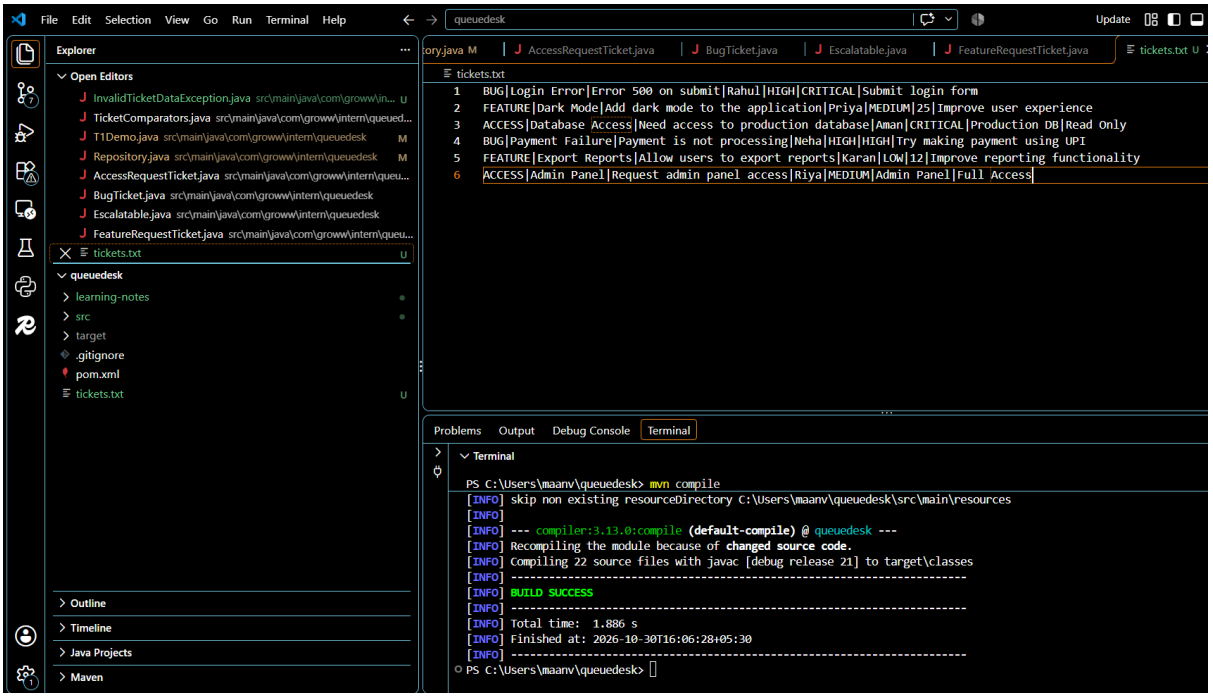
In repository.java

```
//return null; //If no ticket with the given ID is found, returns null.

        //Build T5 Req2: Throw TicketNotFoundException if the ticket
with the given ID is not found.So, throw a custom unchecked exception
instead of returning null.

        throw new TicketNotFoundException("Ticket with ID" + id + "was
not found.");
```

- TicketFileLoader.load(Path file) — reads tickets.txt (pipe-delimited) line by line with try-with-resources over a BufferedReader, using String.split("\\|") and trim() on each field, wrapping any parse failure into InvalidTicketDataException.



Field order

- BUG: Type | Title | Description | Requester | Priority | Severity | Steps
- FEATURE: Type | Title | Description | Requester | Priority | Votes | Business Justification
- ACCESS: Type | Title | Description | Requester | Priority | System Name | Access Level

6 sample tickets - 2 of each type

- TicketFileLoader.load(Path file) — reads tickets.txt (pipe-delimited) line by line with try-with-resources over a BufferedReader, using String.split("\\|") and trim() on each field, wrapping any parse failure into InvalidTicketDataException.

Added TicketFileLoader.java

- A report generator that assembles an end-of-day summary — counts per status and priority, the oldest open ticket, every overdue CRITICAL ticket — using StringBuilder, then writes it with try-with-resources over a BufferedWriter.

Added **TicketReportGeneration.java**

We used `StringBuilder` because the end-of-day report is dynamically generated by combining multiple sections, including ticket counts by status, ticket counts by priority, the oldest OPEN ticket, and all overdue CRITICAL tickets. Since `String` is immutable, repeated concatenation can create multiple temporary String objects. `StringBuilder` is mutable, allowing us to append multiple pieces of text to the same object efficiently. Once the complete report is assembled, we use `toString()` to convert it into a normal String and write it to the output file using `BufferedWriter`.

- Mutable: Its content can be modified without creating a new String for every append operation.
- Performance: More efficient for repeated text concatenation than repeatedly creating new Strings.
- append(): Adds report headings, ticket counts, and ticket details.
- toString(): Converts the completed report into a normal String.
- BufferedWriter: Writes the generated report into a file.
- Why not StringBuffer? StringBuffer synchronizes its methods, whereas our report is built locally by one thread. Therefore, StringBuilder is more suitable and avoids unnecessary synchronization overhead.

- Revisit the T-01 setters: use trim(), blank checks, and safe substring() truncation for long descriptions on the triage board display.

```
//Build-T5 Req6: Returns a shortened description for displaying on the
triage board. If the description is longer than maxLength, it truncates
it and appends "...". If it's shorter or equal, it returns the full
description.
    // Returns a shortened description for displaying on the triage
board.
    public String getDescriptionPreview(int maxLength) {
        // Maximum preview length cannot be negative.
        if (maxLength < 0) {
            throw new IllegalArgumentException("Maximum length cannot
be negative");
        }

        // If description is already within the limit, return it as it
is.
        if (description.length() <= maxLength) {
            return description;
        }

        // substring(0, maxLength) extracts characters from index 0
        // up to maxLength (excluding maxLength).
```

```
        // We append "..." to indicate that the description has been
shortened.
        return description.substring(0, Math.min(maxLength,
description.length())) + "...";
    }
```

T5Demo.java

```
package com.queuedesk;

// Used to represent file paths.
import java.nio.file.Path;

// List stores multiple ticket objects.
import java.util.List;



// Build-T5: Demonstrates loading, exception handling, and report
generation.
public class T5Demo {

    public static void main(String[] args) {

        /*
         * STEP 1: LOAD TICKETS FROM tickets.txt
         *
         * TicketFileLoader.load() reads the file line by line.
         * It returns a List<String[]> containing the fields of each
line.
         */

        Path inputFile = Path.of("tickets.txt");

        try{
            // Load the ticket records from the input file.
            List<String[]> records = TicketFileLoader.load(inputFile);

            // Display how many records were loaded.
            System.out.println("Tickets loaded successfully: " +
records.size());

        }
```

```
        catch (InvalidTicketDataException e) {
            // Handles invalid data found while parsing the file.
            // getMessage() displays the error message, including the
line number.
            System.out.println("Invalid ticket data: " +
e.getMessage());

        }
        catch (java.io.IOException e) {
            // Handles file-related errors such as missing files or
access issues.
            System.out.println("Unable to read tickets file: " +
e.getMessage());
        }



        /*
         * STEP 2: DEMONSTRATE TICKETNOTFOUNDEXCEPTION
         *
         * Repository stores Ticket objects.
         * findById() throws TicketNotFoundException if the requested
ID
         * does not exist in the repository.
         */

        Repository<Ticket> repository = new Repository<>();

        // Add a ticket to the repository.
        Ticket ticket = new FeatureRequestTicket(
                "Login improvement",
                "Improve the login experience",
                "Aman",
                Priority.HIGH,
                5,
                "Improve user experience"
        );

        repository.add(ticket);

        try {
```

```
            // Search for a ticket that does not exist.
            repository.findById(9999);

        }
        catch (TicketNotFoundException e) {

            // Catch the unchecked exception and display a clean
message.
            System.out.println("Repository error: " + e.getMessage());
        }



        /*
         * STEP 3: GENERATE THE END-OF-DAY REPORT
         *
         * TicketReportGenerator generates a summary containing:
         * 1. Counts by status.
         * 2. Counts by priority.
         * 3. The oldest OPEN ticket.
         * 4. Every overdue CRITICAL ticket.
         *
         * The report is saved using BufferedWriter.
         */

        // Specify the location of the output report.
        Path outputFile = Path.of("ticket_report.txt");

        try {

            // Generate and save the report.
            TicketReportGenerator.generateReport(
                    repository.all(),
                    outputFile
            );

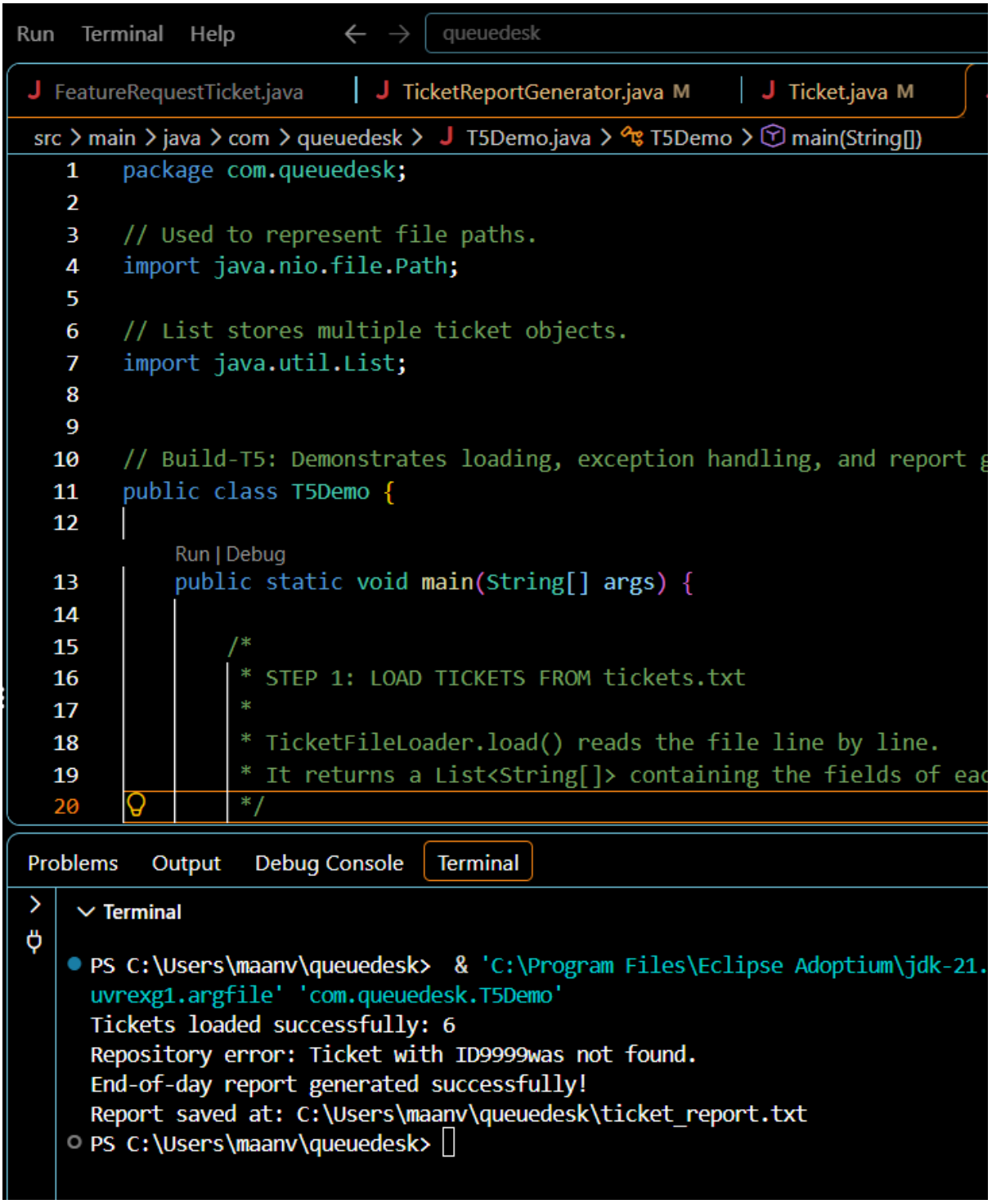
            System.out.println("End-of-day report generated
successfully!");
            System.out.println("Report saved at: " +
outputFile.toAbsolutePath());

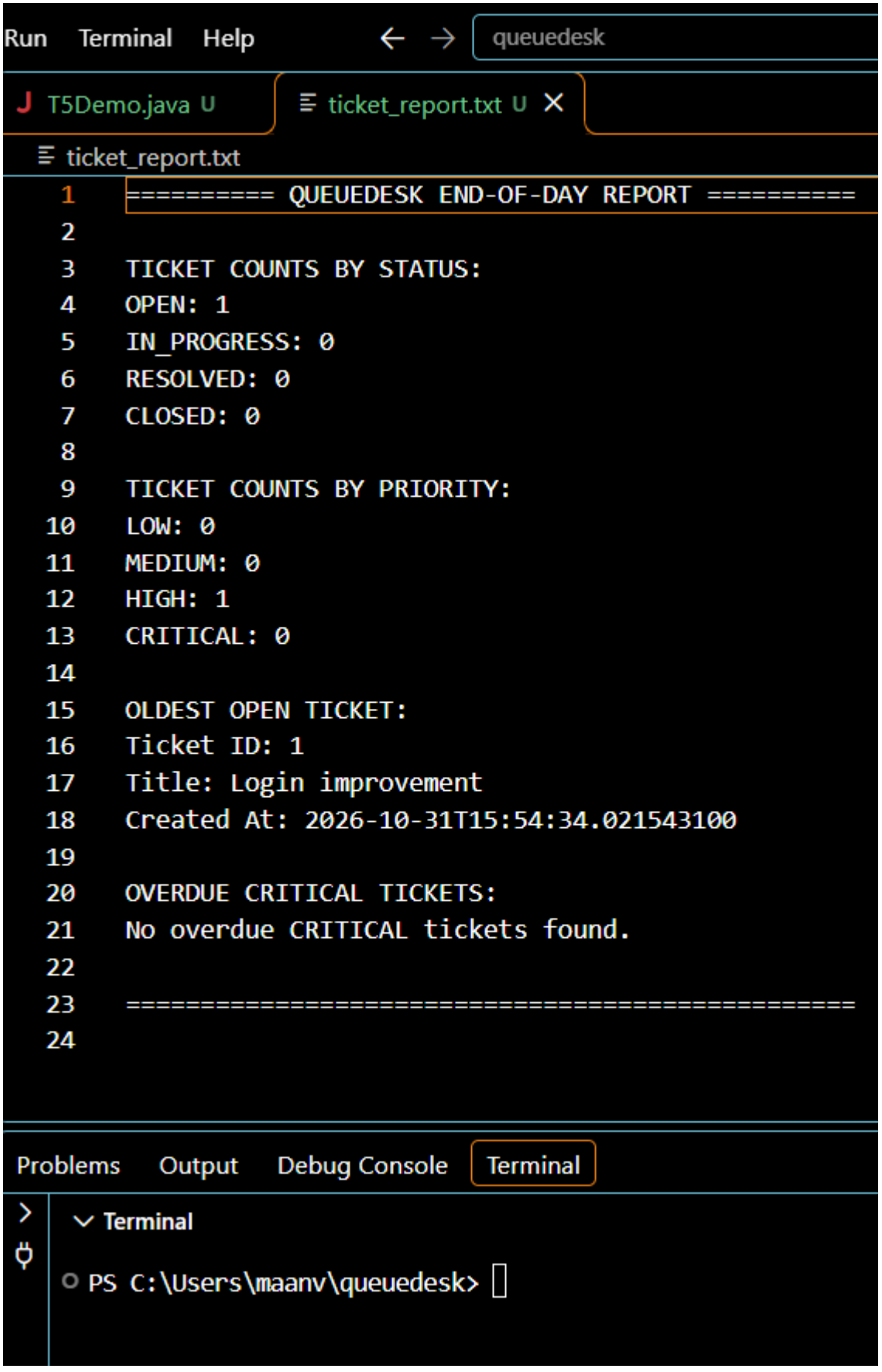
        }
```

```
        catch (java.io.IOException e) {

            // Handle errors that occur while writing the report.
            System.out.println("Unable to generate report: " +
e.getMessage());
        }
    }
}
```

Result:


Run Terminal Help
queuedesk
FeatureRequestTicket.java
TicketReportGenerator.java M
Ticket.java M
src > main > java > com > queuedesk > T5Demo.java > T5Demo > main(String[])
1 package com.queuedesk;
2
3 // Used to represent file paths.
4 import java.nio.file.Path;
5
6 // List stores multiple ticket objects.
7 import java.util.List;
8
9
10 // Build-T5: Demonstrates loading, exception handling, and report g
11 public class T5Demo {
12
Run | Debug
13 public static void main(String[] args) {
14
15 /*
16 * STEP 1: LOAD TICKETS FROM tickets.txt
17 *
18 * TicketFileLoader.load() reads the file line by line.
19 * It returns a List<String[]> containing the fields of eac
20 */
Problems Output Debug Console Terminal
Terminal
PS C:\Users\maanv\queuedesk> & 'C:\Program Files\Eclipse Adoptium\jdk-21.
uvrexg1.argfile' 'com.queuedesk.T5Demo'
Tickets loaded successfully: 6
Repository error: Ticket with ID9999was not found.
End-of-day report generated successfully!
Report saved at: C:\Users\maanv\queuedesk\ticket_report.txt
PS C:\Users\maanv\queuedesk>

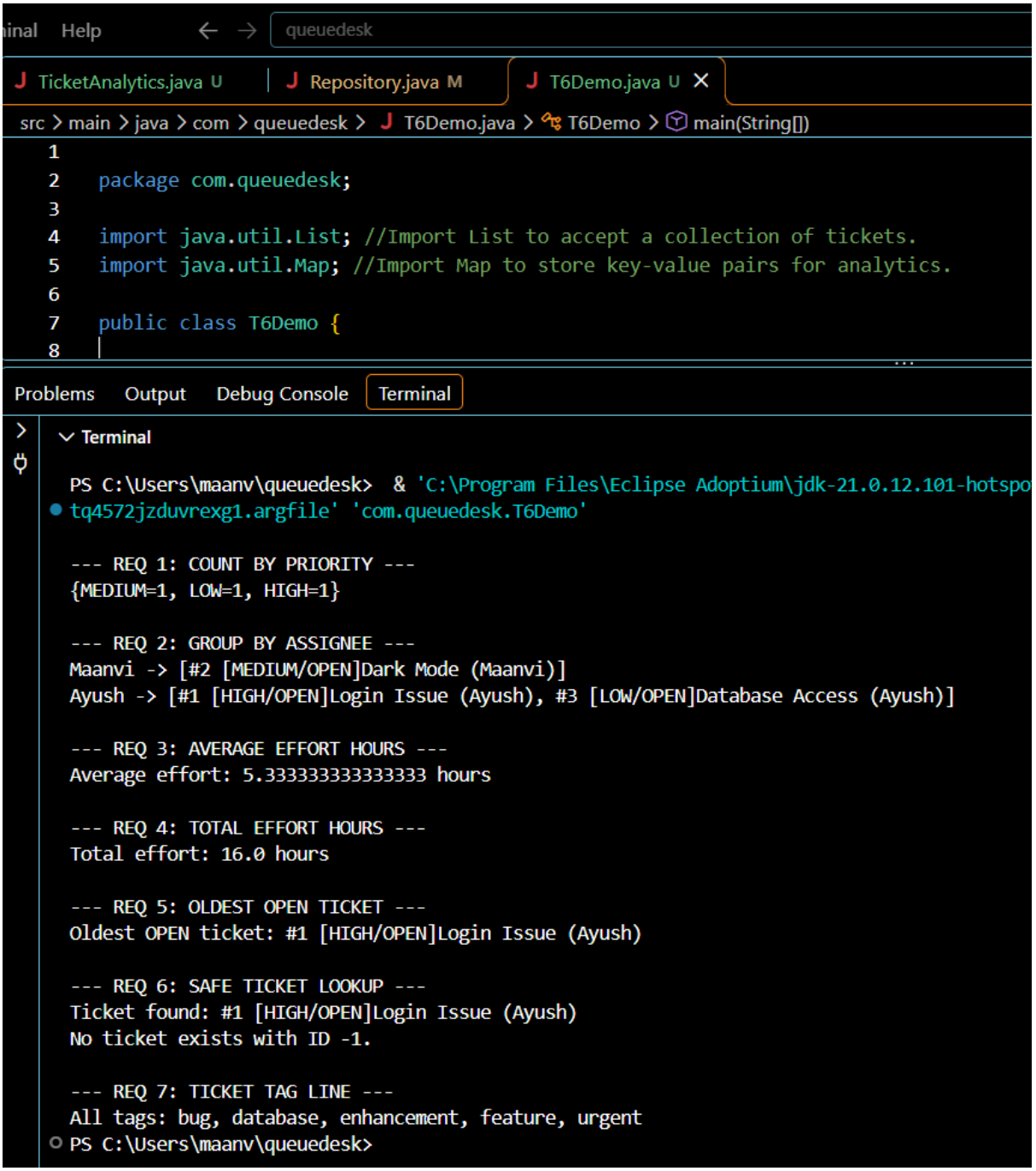

Run Terminal Help
queuedesk
T5Demo.java U
ticket_report.txt U
ticket_report.txt
1 ========== QUEUEDESK END-OF-DAY REPORT ==========
2
3 TICKET COUNTS BY STATUS:
4 OPEN: 1
5 IN_PROGRESS: 0
6 RESOLVED: 0
7 CLOSED: 0
8
9 TICKET COUNTS BY PRIORITY:
10 LOW: 0
11 MEDIUM: 0
12 HIGH: 1
13 CRITICAL: 0
14
15 OLDEST OPEN TICKET:
16 Ticket ID: 1
17 Title: Login improvement
18 Created At: 2026-10-31T15:54:34.021543100
19
20 OVERDUE CRITICAL TICKETS:
21 No overdue CRITICAL tickets found.
22
23 ==
24
Problems Output Debug Console Terminal
Terminal
PS C:\Users\maanv\queuedesk>

Build-T6

- Build: TicketAnalytics
  - Map<Priority, Long> countByPriority(List<Ticket> tickets) — stream().collect(Collectors.groupingBy(Ticket::getPriority, Collectors.counting())).
  - Map<String, List<Ticket>> groupByAssignee(List<Ticket> tickets) — rewrite the T-03 loop version as a one-line Collectors.groupingBy stream.
  - double averageEffortHours(List<Ticket> tickets) — via mapToDouble(Ticket::estimateEffortHours).average(), handling the empty-list case explicitly.
  - double totalEffortHours(List<Ticket> tickets) — the same total from T-05, this time via .stream().reduce(0.0, Double::sum, Double::sum), so the two approaches sit side by side for comparison.
  - Optional<Ticket> oldestOpenTicket(List<Ticket> tickets) — filter to OPEN, then min(Comparator.comparing(Ticket::getCreatedAt)), which already returns Optional<Ticket>.
  - Add Optional<Ticket> Repository<T>.findByIdSafe(int id) alongside the existing T-04 findById that throws — same lookup, two calling conventions. Use it with .map(...), .ifPresentOrElse(...), and .orElseThrow(() → new TicketNotFoundException(id)) in different call sites, matching the convention to the caller's needs.
  - A String tagLine(List<Ticket> tickets) using flatMap over each ticket's tag Set<String>, distinct(), sorted(), and Collectors.joining(", ").

Added **TicketAnalytics.java**
Req1,2,3,4,5,7
Added Req6 i**n Repository.java**

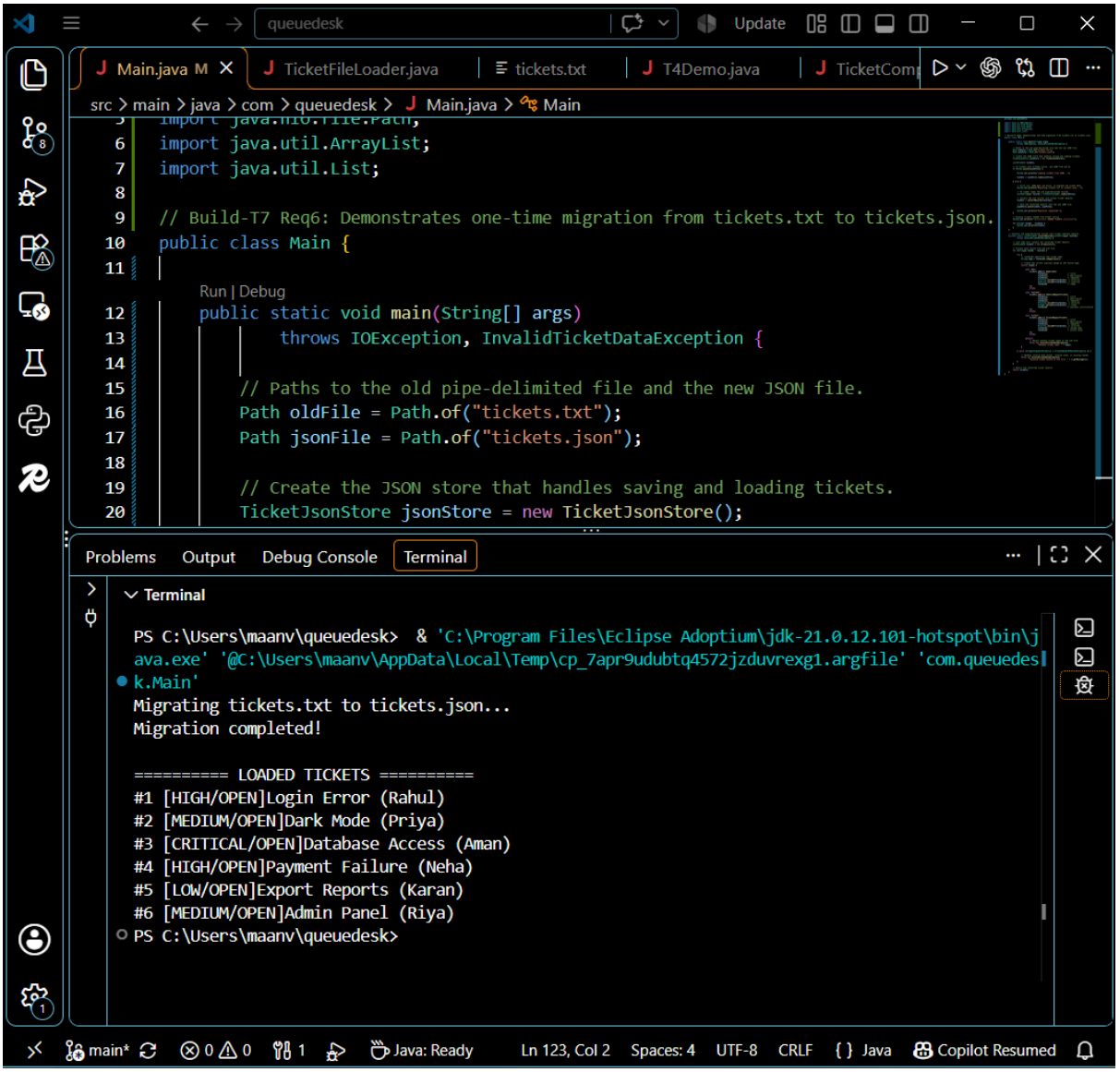
T6Demo.java

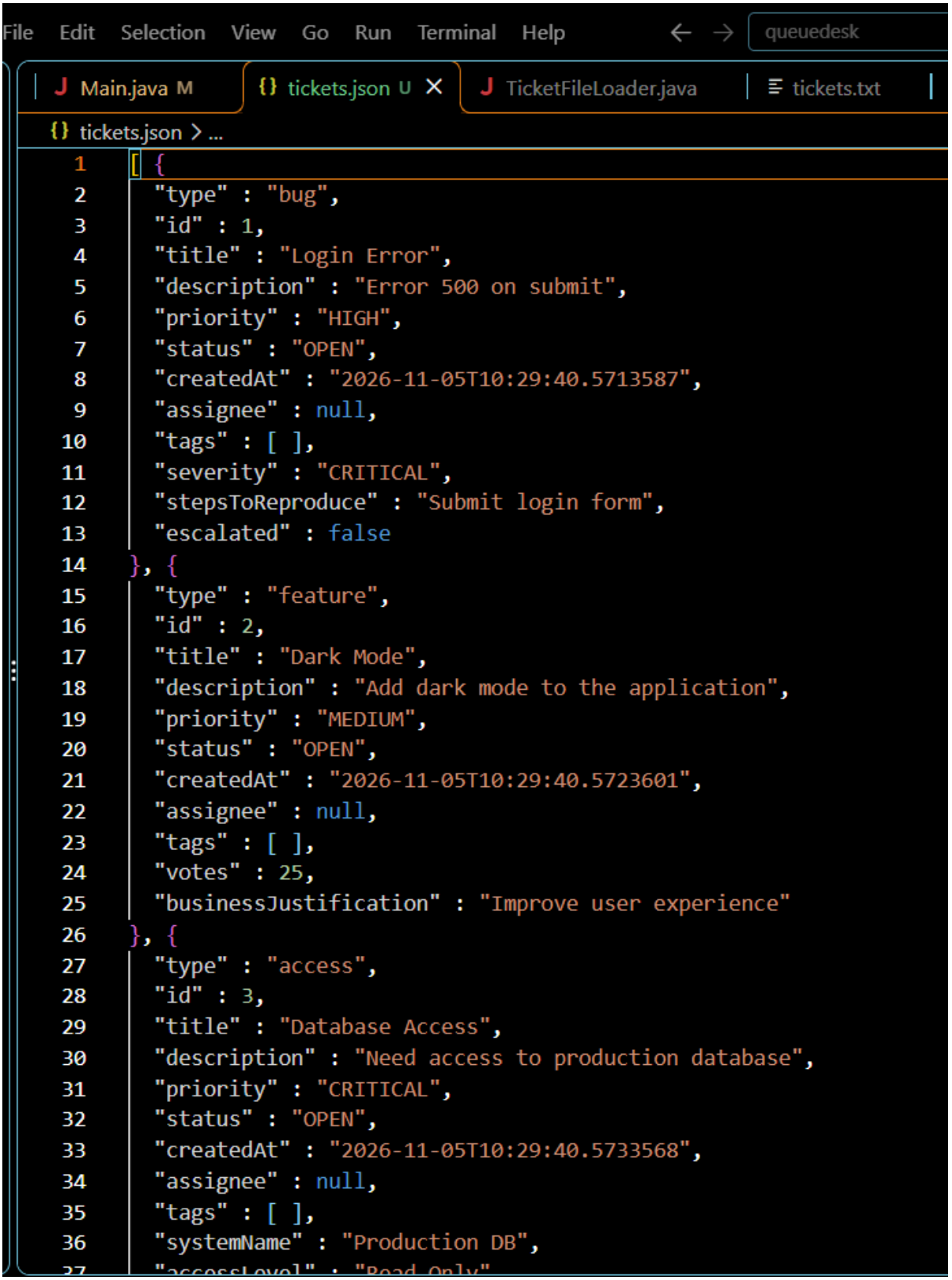

inal Help
queuedesk
TicketAnalytics.java U
Repository.java M
T6Demo.java U
src > main > java > com > queuedesk > T6Demo.java > T6Demo > main(String[])
1
2 package com.queuedesk;
3
4 import java.util.List; //Import List to accept a collection of tickets.
5 import java.util.Map; //Import Map to store key-value pairs for analytics.
6
7 public class T6Demo {
8
Problems
Output
Debug Console
Terminal
Terminal
PS C:\Users\maanv\queuedesk> & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.12.101-hotspo
tq4572jzduvrexg1.argfile' 'com.queuedesk.T6Demo'
--- REQ 1: COUNT BY PRIORITY ---
{MEDIUM=1, LOW=1, HIGH=1}
--- REQ 2: GROUP BY ASSIGNEE ---
Maanvi -> [#2 [MEDIUM/OPEN]Dark Mode (Maanvi)]
Ayush -> [#1 [HIGH/OPEN]Login Issue (Ayush), #3 [LOW/OPEN]Database Access (Ayush)]
--- REQ 3: AVERAGE EFFORT HOURS ---
Average effort: 5.333333333333333 hours
--- REQ 4: TOTAL EFFORT HOURS ---
Total effort: 16.0 hours
--- REQ 5: OLDEST OPEN TICKET ---
Oldest OPEN ticket: #1 [HIGH/OPEN]Login Issue (Ayush)
--- REQ 6: SAFE TICKET LOOKUP ---
Ticket found: #1 [HIGH/OPEN]Login Issue (Ayush)
No ticket exists with ID -1.
--- REQ 7: TICKET TAG LINE ---
All tags: bug, database, enhancement, feature, urgent
PS C:\Users\maanv\queuedesk>

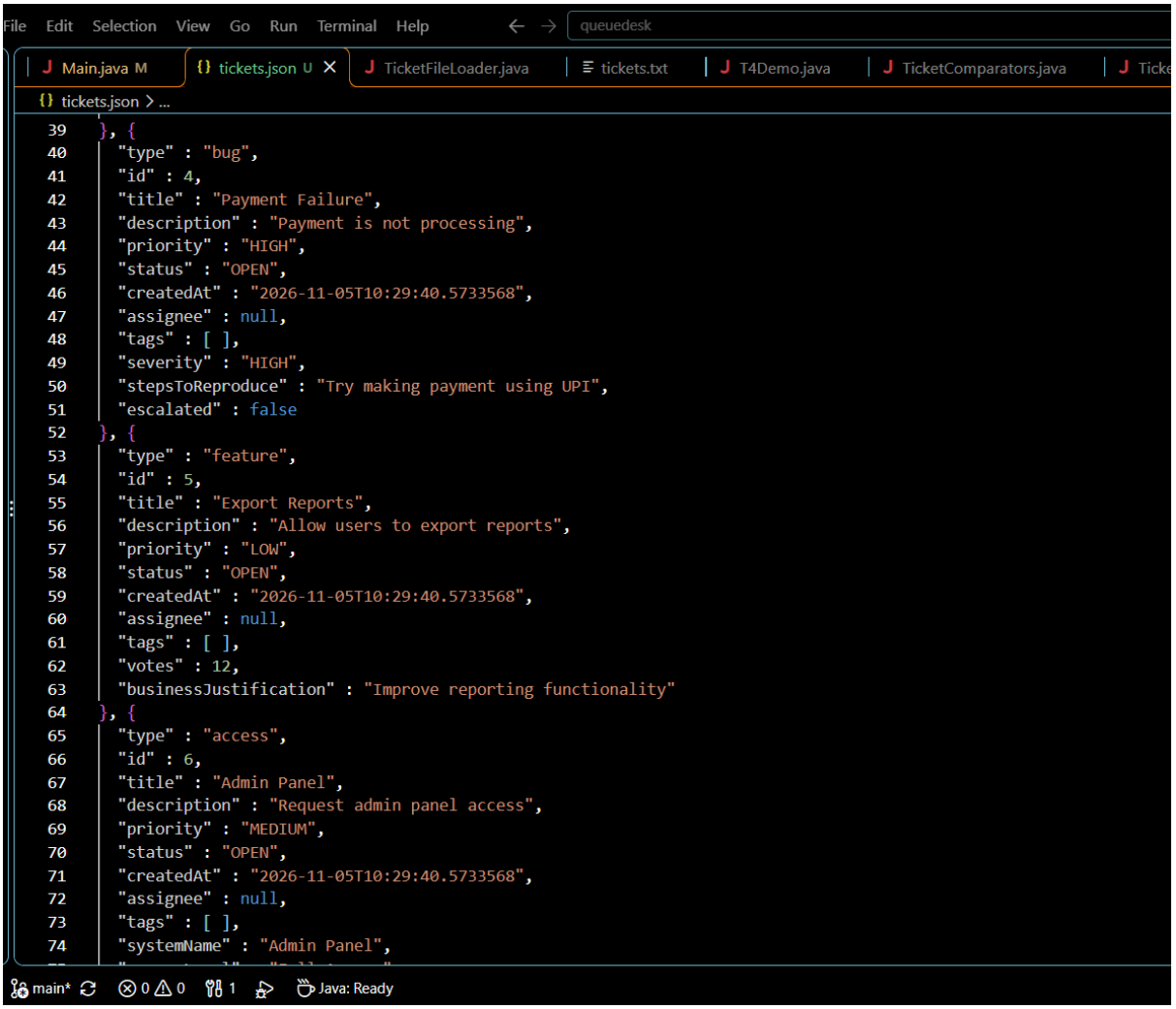
Build-T7

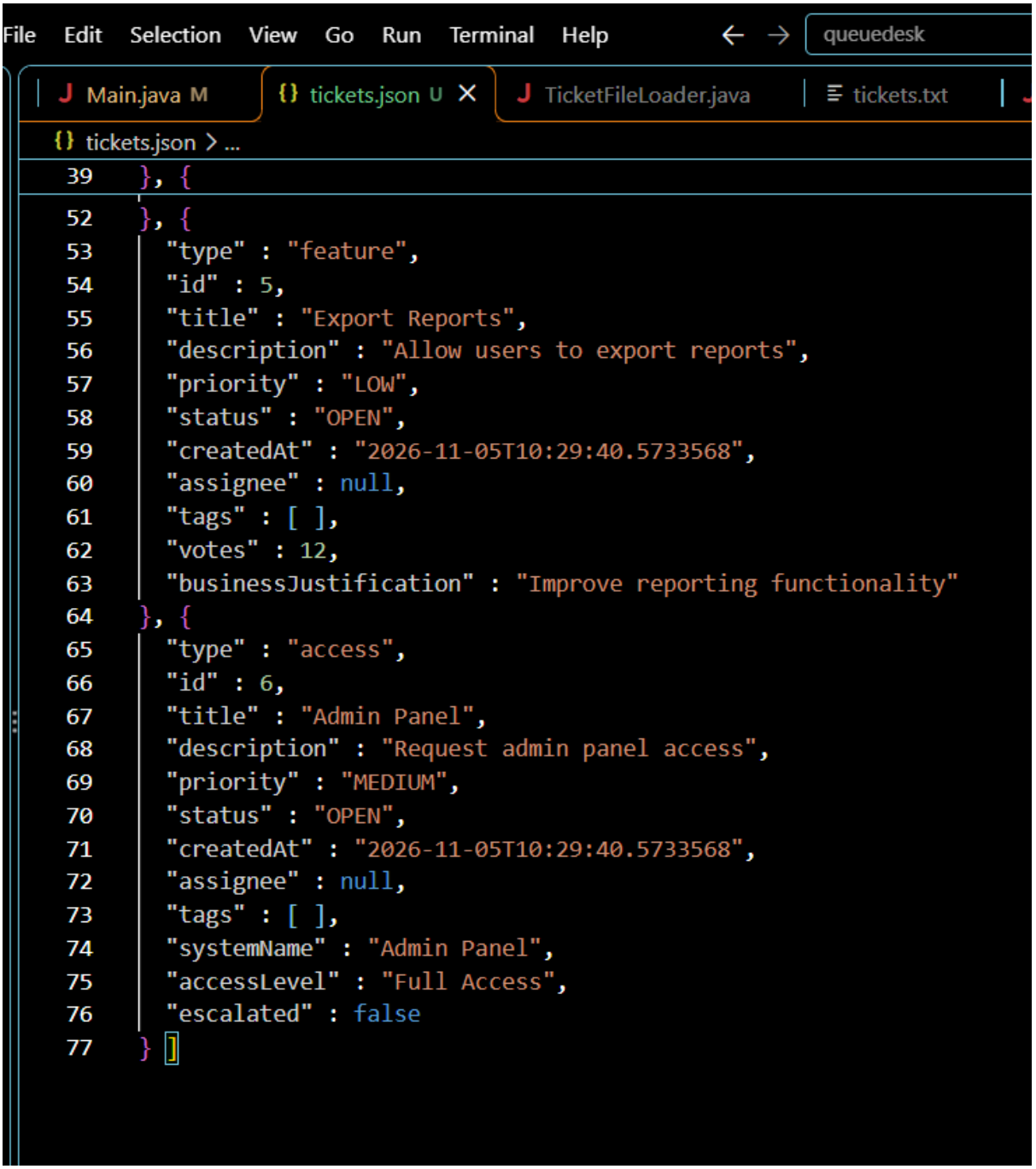
- Build: tickets.json
  - Add @JsonTypeInfo(use = JsonTypeInfo.Id.NAME, include = JsonTypeInfo.As.PROPERTY, property = "type") and @JsonSubTypes to Ticket, naming each subtype ("bug", "feature", "access").
  - Build the mapper once, in one place: new ObjectMapper().registerModule(new JavaTimeModule()).disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS).enable(SerializationFeature.INDENT_OUTPUT).
  - TicketJsonStore.save(List<Ticket> tickets, Path file) — mapper.writeValue(file.toFile(), tickets).
  - List<Ticket> TicketJsonStore.load(Path file) — deserialize with a TypeReference<List<Ticket>>, so subtypes come back as their real classes, not generic maps.
  - Wrap Jackson's checked IOException/JsonProcessingException into your existing InvalidTicketDataException at the store's boundary — callers above this layer shouldn't need to know Jackson is involved.

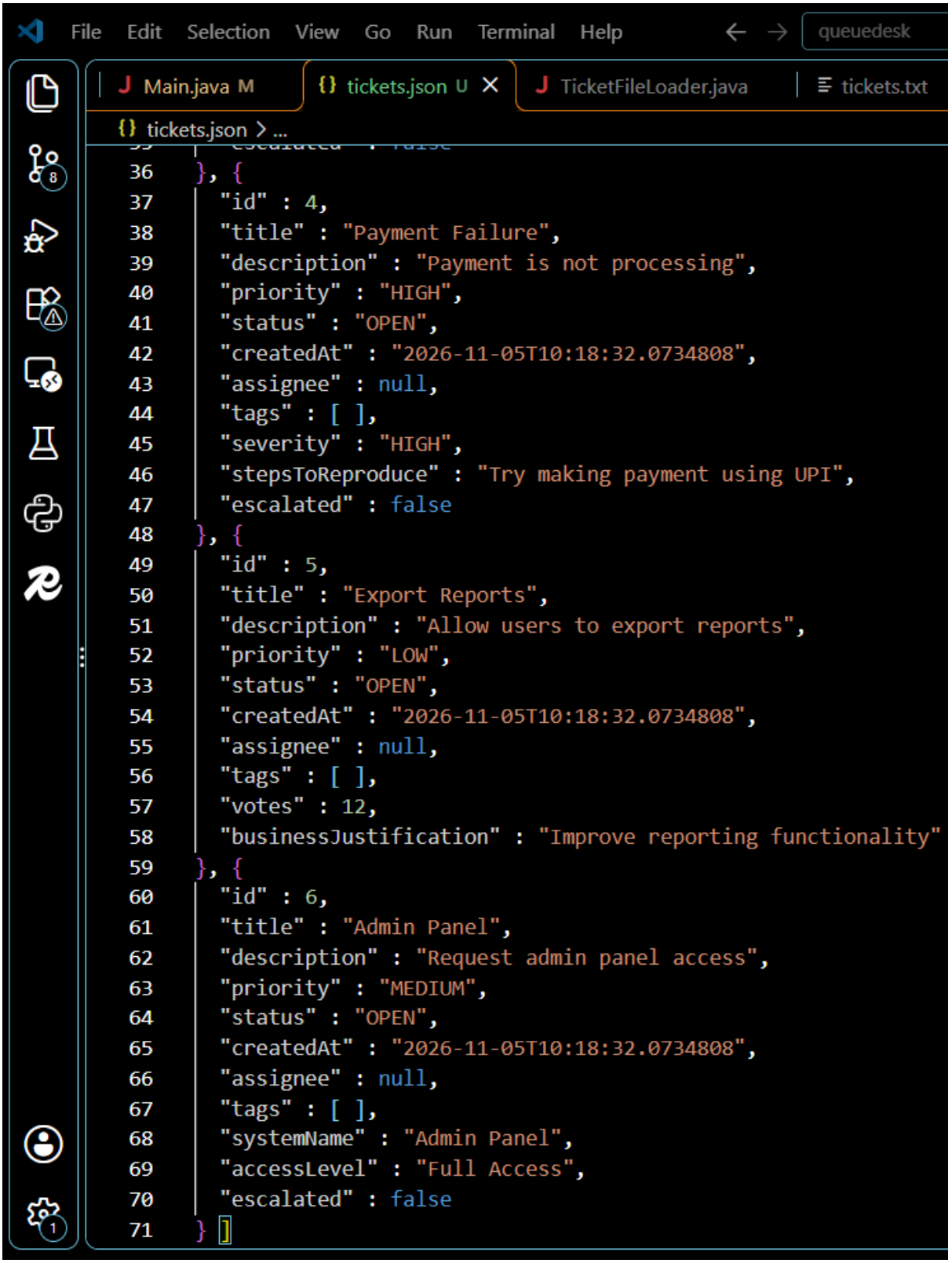
  - Keep the T-04 pipe-delimited loader working as a one-time migration path: read the old format, write it out once as tickets.json, and use JSON as the format from then on.

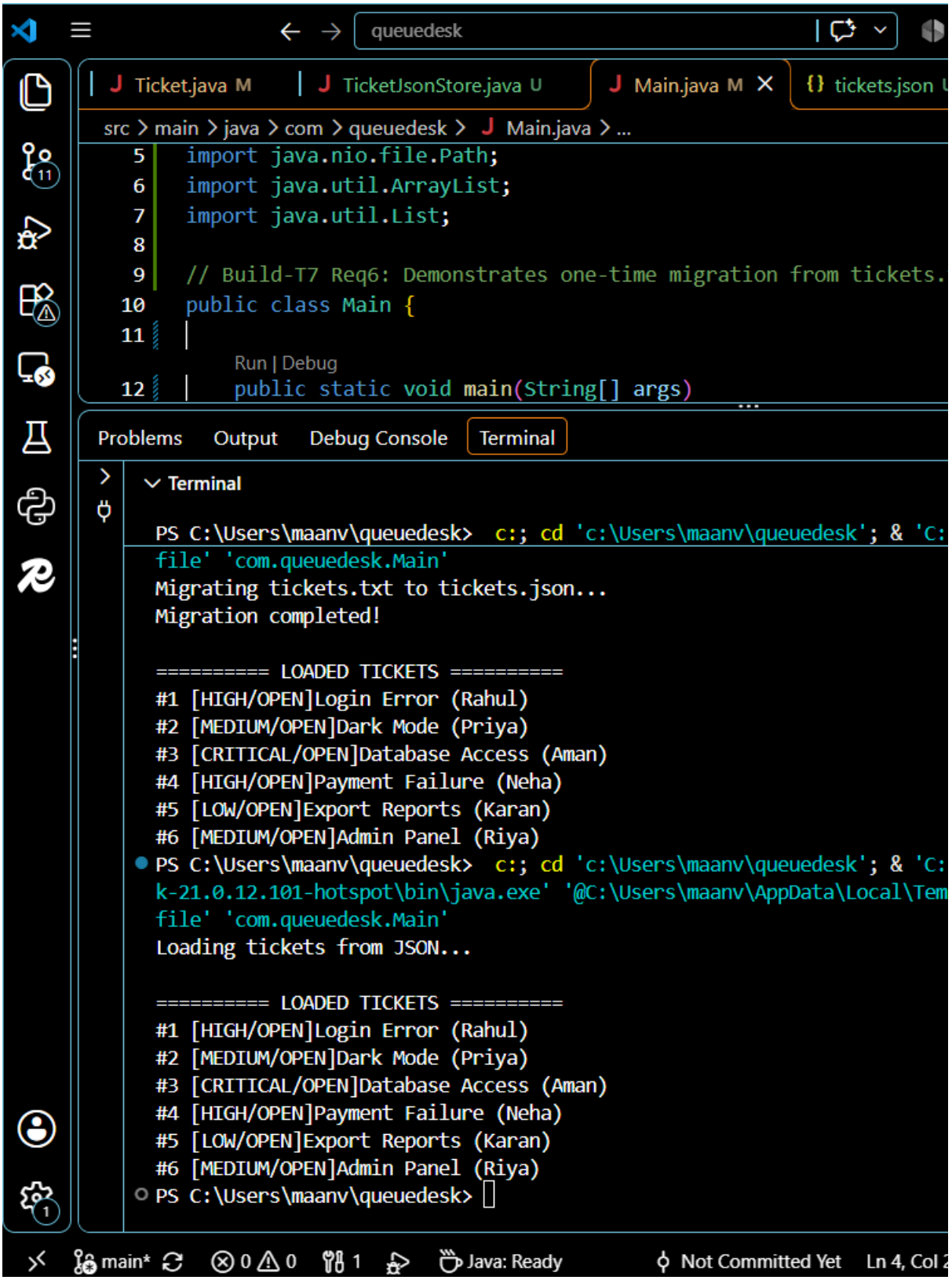

queuedesk
Update
Main.java M
TicketFileLoader.java
tickets.txt
T4Demo.java
src > main > java > com > queuedesk > Main.java > Main
import java.util.ArrayList;
import java.util.List;
// Build-T7 Req6: Demonstrates one-time migration from tickets.txt to tickets.json.
public class Main {
Run | Debug
public static void main(String[] args)
throws IOException, InvalidTicketDataException {
// Paths to the old pipe-delimited file and the new JSON file.
Path oldFile = Path.of("tickets.txt");
Path jsonFile = Path.of("tickets.json");
// Create the JSON store that handles saving and loading tickets.
TicketJsonStore jsonStore = new TicketJsonStore();
Problems
Output
Debug Console
Terminal
Terminal
PS C:\Users\maanv\queuedesk> & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.12.101-hotspot\bin\java.exe' '@C:\Users\maanv\AppData\Local\Temp\cp_7apr9ududbtq4572jzduvrexg1.argfile' 'com.queuedesk.Main'
Migrating tickets.txt to tickets.json...
Migration completed!
========== LOADED TICKETS ==========
#1 [HIGH/OPEN]Login Error (Rahul)
#2 [MEDIUM/OPEN]Dark Mode (Priya)
#3 [CRITICAL/OPEN]Database Access (Aman)
#4 [HIGH/OPEN]Payment Failure (Neha)
#5 [LOW/OPEN]Export Reports (Karan)
#6 [MEDIUM/OPEN]Admin Panel (Riya)
PS C:\Users\maanv\queuedesk>
main*
Java: Ready
Ln 123, Col 2
Spaces: 4
UTF-8
CRLF
Java
Copilot Resumed


File Edit Selection View Go Run Terminal Help
queuedesk
Main.java M
tickets.json U
TicketFileLoader.java
tickets.txt
tickets.json > ...
[{
"type" : "bug",
"id" : 1,
"title" : "Login Error",
"description" : "Error 500 on submit",
"priority" : "HIGH",
"status" : "OPEN",
"createdAt" : "2026-11-05T10:29:40.5713587",
"assignee" : null,
"tags" : [ ],
"severity" : "CRITICAL",
"stepsToReproduce" : "Submit login form",
"escalated" : false
}, {
"type" : "feature",
"id" : 2,
"title" : "Dark Mode",
"description" : "Add dark mode to the application",
"priority" : "MEDIUM",
"status" : "OPEN",
"createdAt" : "2026-11-05T10:29:40.5723601",
"assignee" : null,
"tags" : [ ],
"votes" : 25,
"businessJustification" : "Improve user experience"
}, {
"type" : "access",
"id" : 3,
"title" : "Database Access",
"description" : "Need access to production database",
"priority" : "CRITICAL",
"status" : "OPEN",
"createdAt" : "2026-11-05T10:29:40.5733568",
"assignee" : null,
"tags" : [ ],
"systemName" : "Production DB",


File Edit Selection View Go Run Terminal Help
queuedesk
Main.java M
tickets.json U
TicketFileLoader.java
tickets.txt
T4Demo.java
TicketComparators.java
tickets.json > ...
39 }, {
40 "type" : "bug",
41 "id" : 4,
42 "title" : "Payment Failure",
43 "description" : "Payment is not processing",
44 "priority" : "HIGH",
45 "status" : "OPEN",
46 "createdAt" : "2026-11-05T10:29:40.5733568",
47 "assignee" : null,
48 "tags" : [ ],
49 "severity" : "HIGH",
50 "stepsToReproduce" : "Try making payment using UPI",
51 "escalated" : false
52 }, {
53 "type" : "feature",
54 "id" : 5,
55 "title" : "Export Reports",
56 "description" : "Allow users to export reports",
57 "priority" : "LOW",
58 "status" : "OPEN",
59 "createdAt" : "2026-11-05T10:29:40.5733568",
60 "assignee" : null,
61 "tags" : [ ],
62 "votes" : 12,
63 "businessJustification" : "Improve reporting functionality"
64 }, {
65 "type" : "access",
66 "id" : 6,
67 "title" : "Admin Panel",
68 "description" : "Request admin panel access",
69 "priority" : "MEDIUM",
70 "status" : "OPEN",
71 "createdAt" : "2026-11-05T10:29:40.5733568",
72 "assignee" : null,
73 "tags" : [ ],
74 "systemName" : "Admin Panel",
main*
0 0
1
Java: Ready


File Edit Selection View Go Run Terminal Help
queuedesk
Main.java M
tickets.json U
TicketFileLoader.java
tickets.txt
tickets.json > ...
39 }, {
52 }, {
53 "type" : "feature",
54 "id" : 5,
55 "title" : "Export Reports",
56 "description" : "Allow users to export reports",
57 "priority" : "LOW",
58 "status" : "OPEN",
59 "createdAt" : "2026-11-05T10:29:40.5733568",
60 "assignee" : null,
61 "tags" : [ ],
62 "votes" : 12,
63 "businessJustification" : "Improve reporting functionality"
64 }, {
65 "type" : "access",
66 "id" : 6,
67 "title" : "Admin Panel",
68 "description" : "Request admin panel access",
69 "priority" : "MEDIUM",
70 "status" : "OPEN",
71 "createdAt" : "2026-11-05T10:29:40.5733568",
72 "assignee" : null,
73 "tags" : [ ],
74 "systemName" : "Admin Panel",
75 "accessLevel" : "Full Access",
76 "escalated" : false
77 }]


File Edit Selection View Go Run Terminal Help
queuedesk
Main.java M
tickets.json U
TicketFileLoader.java
tickets.txt
tickets.json > ...
36 }, {
37 "id" : 4,
38 "title" : "Payment Failure",
39 "description" : "Payment is not processing",
40 "priority" : "HIGH",
41 "status" : "OPEN",
42 "createdAt" : "2026-11-05T10:18:32.0734808",
43 "assignee" : null,
44 "tags" : [ ],
45 "severity" : "HIGH",
46 "stepsToReproduce" : "Try making payment using UPI",
47 "escalated" : false
48 }, {
49 "id" : 5,
50 "title" : "Export Reports",
51 "description" : "Allow users to export reports",
52 "priority" : "LOW",
53 "status" : "OPEN",
54 "createdAt" : "2026-11-05T10:18:32.0734808",
55 "assignee" : null,
56 "tags" : [ ],
57 "votes" : 12,
58 "businessJustification" : "Improve reporting functionality"
59 }, {
60 "id" : 6,
61 "title" : "Admin Panel",
62 "description" : "Request admin panel access",
63 "priority" : "MEDIUM",
64 "status" : "OPEN",
65 "createdAt" : "2026-11-05T10:18:32.0734808",
66 "assignee" : null,
67 "tags" : [ ],
68 "systemName" : "Admin Panel",
69 "accessLevel" : "Full Access",
70 "escalated" : false
71 }]


queuedesk
Ticket.java M
TicketJsonStore.java U
Main.java M
tickets.json
src > main > java > com > queuedesk > Main.java > ...
5 import java.nio.file.Path;
6 import java.util.ArrayList;
7 import java.util.List;
8
9 // Build-T7 Req6: Demonstrates one-time migration from tickets.
10 public class Main {
11
Run | Debug
12 public static void main(String[] args)
Problems
Output
Debug Console
Terminal
Terminal
PS C:\Users\maanv\queuedesk> c:; cd 'c:\Users\maanv\queuedesk'; & 'C:
file' 'com.queuedesk.Main'
Migrating tickets.txt to tickets.json...
Migration completed!
========== LOADED TICKETS ==========
#1 [HIGH/OPEN]Login Error (Rahul)
#2 [MEDIUM/OPEN]Dark Mode (Priya)
#3 [CRITICAL/OPEN]Database Access (Aman)
#4 [HIGH/OPEN]Payment Failure (Neha)
#5 [LOW/OPEN]Export Reports (Karan)
#6 [MEDIUM/OPEN]Admin Panel (Riya)
PS C:\Users\maanv\queuedesk> c:; cd 'c:\Users\maanv\queuedesk'; & 'C:
k-21.0.12.101-hotspot\bin\java.exe' '@C:\Users\maanv\AppData\Local\Tem
file' 'com.queuedesk.Main'
Loading tickets from JSON...
========== LOADED TICKETS ==========
#1 [HIGH/OPEN]Login Error (Rahul)
#2 [MEDIUM/OPEN]Dark Mode (Priya)
#3 [CRITICAL/OPEN]Database Access (Aman)
#4 [HIGH/OPEN]Payment Failure (Neha)
#5 [LOW/OPEN]Export Reports (Karan)
#6 [MEDIUM/OPEN]Admin Panel (Riya)
PS C:\Users\maanv\queuedesk>
main*
0 0
1
Java: Ready
Not Committed Yet
Ln 4, Col 2